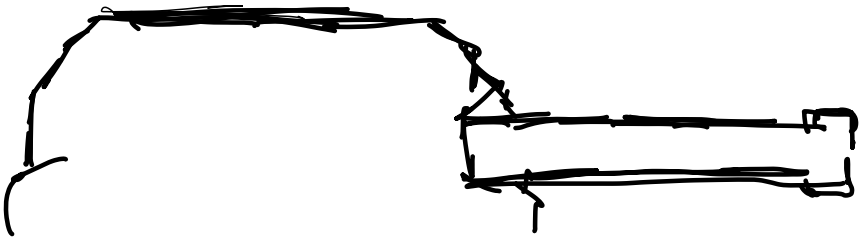


August 24, 2026

ADT LIST

Activity (In Room)

(A) Look for 6 people

	(String)	(String)	(String)	(char)	(int)
no.	Name	id	course	gender	age
1	Amara		BSIT	M	21
2	Patrishia				
3	Chelsy				
4	Danniah				
5	Kurt				
6	Janie				

(B) Insert 1 person (from your seatmate)

no.	Name
1	Thea
2	Amara
3	Patrishia
4	Chelsy
5	Danniah
6	Kurt
7	Janie

(C) Sort the List

no.	Name
1	Amara
2	Chelsy
3	Danniah
4	Janie
5	Kurt
6	Patrishia
7	Thea

(D) Delete the 4th position

no.	Name
1	Amara
2	Chelsy
3	Danniah
4	Kurt
5	Patrishia
6	Thea

September 2, 2026

BIG O NOTATION

RECAP

```
header (          ){
```

error catching { parameter, isEmpty, isFull

```
} else {
```

process { for ( ) / while ( ) (for/while loops)

updating { lastNdx ++; / return A;

CONCEPT:

Let's say from USC you wanna go to Ayala Center Block (ACB). Which of these routes with the given estimated time you'd arrive the fastest but also less hassle?

USC ⟶ ACB

Task:

walk == ₱0 -> cost
1hr -> arrival

jeep == ₱15
== 20 mins

angkas == ~~₱~~ 65
== 10 mins

This is similar to the Big O Notation. Depending on how you code your program it's either: it will take time to run or too fast to run.

CODING PREFERENCE:

For example we have 3 for loops

```
for ( n) {
    for ( n) {
        for ( n) {

        }
    }
}
```

// iterates the list
// checks the elements (?)
// shifting

EXAMPLE

```
for ( 10 ) {
    for ( 5 ) {
        for ( 3 ) {

        }
    }
}
```

150 = 10 × 5 × 3 (n~)

it will run 15 times but iterates for 150 times

15 < 150

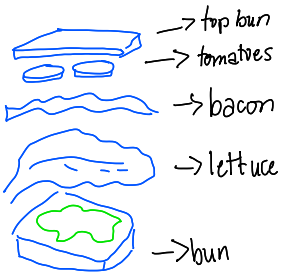
$n * n * n == n^3$

$> O(n^2)$

September 2, 2026

STACK

Let's say we want to create a sandwich:



When you make a sandwich you start with the bottom ingredient and slowly go up. Now let's say you wanna add mayonnaise, you remove everything starting from the top and the bottom will be last.

STACK OPERATIONS

```
typedef struct {
    String name;
    String specie;
    String color;
    int breadi;
    int age;
} Bird;
```

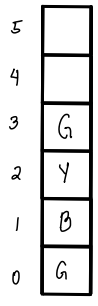
```
typedef struct {
    Bird breadBirds[10];
    int top;
} Stack;
```

void push → insert
void pop → delete
Bird peek → retrieving the specific data

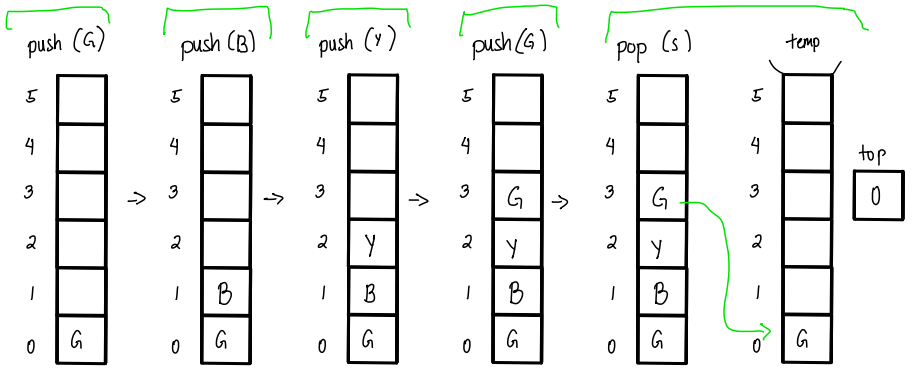
top

Memory of array

G B Y G




```
pop (s, x)
peek (s) == G
pop (s)
push (temp s, x)
```



OPERATIONS CONCEPT

Array

```
push (s, x) {
    if( !isFull) {
        s.data[s.top++] = x;
    } else {
        printf("Stack is full");
    }
    return s;
}
```

Ptr

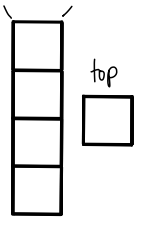
```
push (s, x) {
    if( !isFull) {
        s-> data[s->top++] = x;
    } else {
        printf("Stack is full");
    }
    return s;
}
```

```
pop(s) {
    if (!isFull) {
        s.top--;
    } else {
        printf("Stack is empty");
    }
```

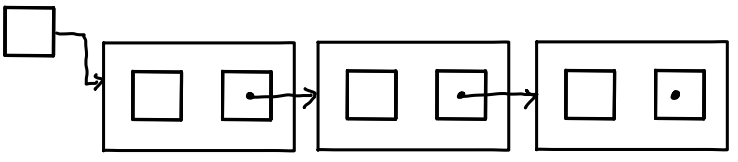
```
pop(s) {
    if (!isFull) {
        s->top--;
    } else {
        printf("Stack is empty");
    }
```

```
typedef struct {
    Birds breadBirds[10];
    int top;
} Stack;
```

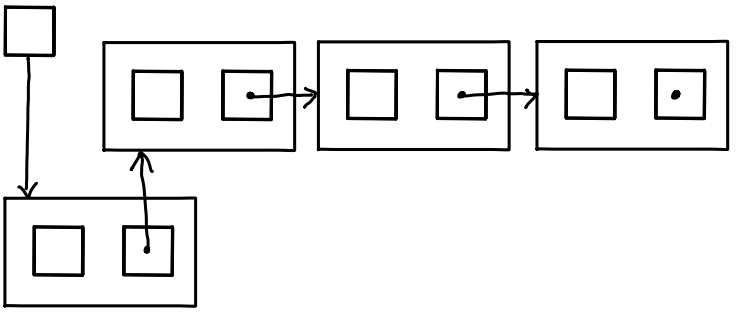
ARRAY



LINKED LIST



OR when inserting a new node:



LIST	Last	First
Array	O(1)	O(n)
Linked List	O(n)	O(1)

ASSIGNMENTS / ACTIVITIES

1.) Linked List activity based on angels

2.) Quiz Gemini

3.) Coding Assignment (Stack)

↳ 1.) push (void), pop, peek (Birds)

only use stack operations → 2.) pushUnique

only use stack operations → 3.) countStack — returns int

Sept. 7, 2026

CIS 2101

STACK pt. 2

(Answering of the assignment)

```
typedef struct {
    String name;
    String specie;
    String color;
    int breed;
    int age;
}
```

Stack:
→ use operations

dili ka access index sa stack unless stated otherwise.

```
typedef struct {
    Bird breedBirds[10];
    int top;
} Stack;
```

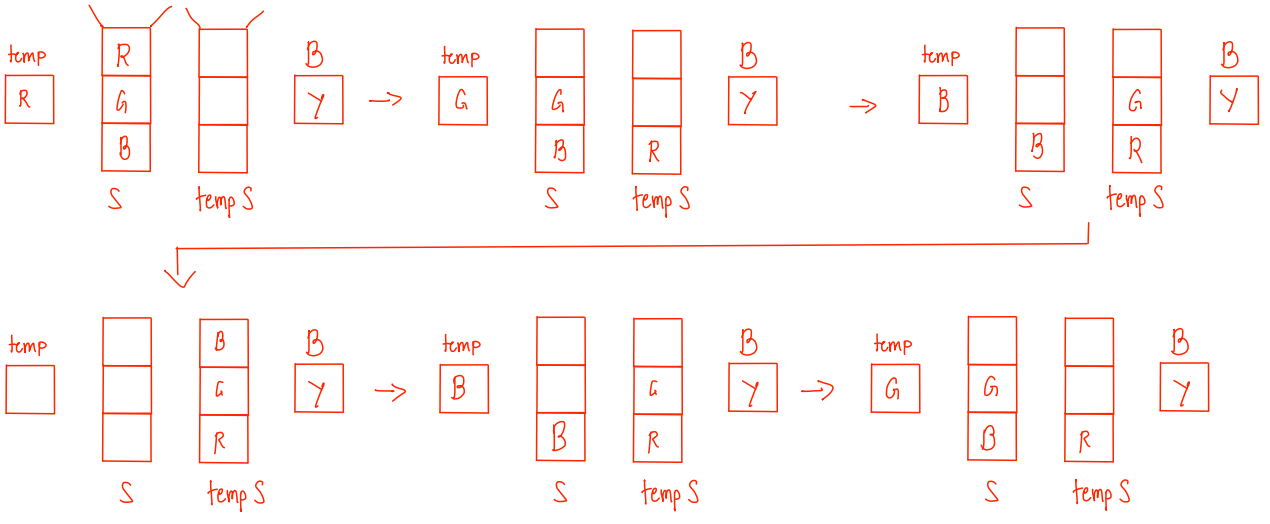
```
void pushUnique(Stack *S, Bird B) {
    if ( S->top > 10) {          // isfull
        printf("Full");
    } else {
        while ( S->top ≥ 0) {    // isEmpty
    Stack? ⇒ temp = peek(S);
what data type? if ( temp -> color != B.color ) {
                push (temp S, temp);   // temp → what's this
                pop (S);
            } else {
                break;
            }
```

note: no need to return since its void

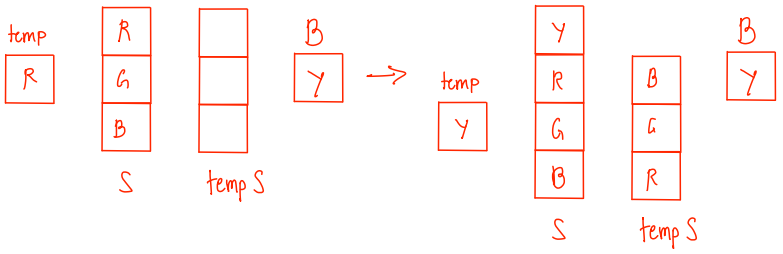
```
void pushUnique(Stack *S, Bird B) {
    if ( S->top > 10) {          // isFull
        printf("Full");
    } else {
        while ( S->top >= 0 &&   // isEmpty
                strcmp(temp->color, B.color) != 0) {
            pop(S);
            push(temp S, temp);
            temp = peek(S);
        }                        // iteration for stack

    if (isEmpty(S)) {
        while (!isEmpty(temp S)) {
            push(S, peek(temp S));
            pop(temp S);
        }
        push(S, B);
    } else {
        while (!isEmpty(temp S)) {   // (since its a pointer return everything to its original state.)
            push(S, peek(temp S));
            pop(temp S);
        }
        push(S, B);
        printf("duplicate found");
    }                            // I will fix this in the future :)
```

DRAWING FOR THIS PART CAN BE FOUND BELOW

COLOR PATTERN: R -> G -> B -> Y



something ?



Count Stack → only do the operations.

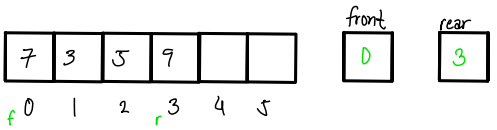
```
int countStack (S) {
    int retVal = 0;
    while (!isEmpty (S)){
        retVal++;
    }
    return retVal;
}
```

← after count Stack is done

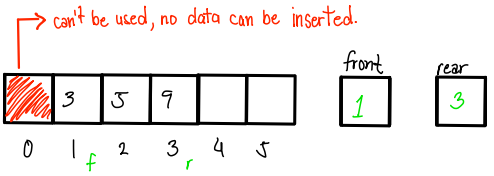
Sept. 7, 2026

QUEUE

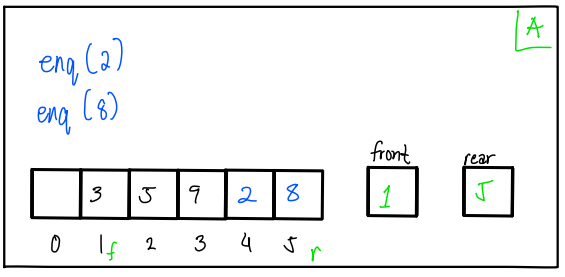
ARRAY IMPLEMENTATION

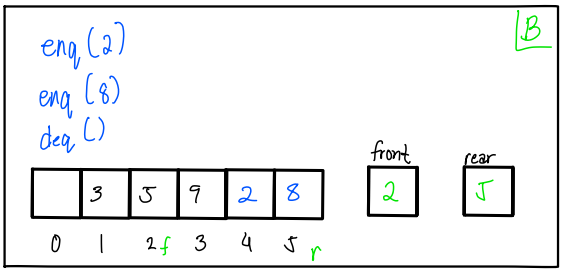


enq(7)
enq(3)
enq(5)
enq(9)
deq( ) →



Now let's say you wanna add the data values: 2 & 8, you have to increment the rear





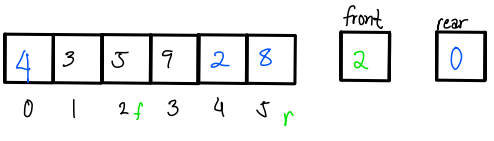
- Now the array is full. We want to insert 4. In order to put the value 4 at index 0, we will use modullos. Since the size of the array is 6, we will divide 4 by 6. Such that:

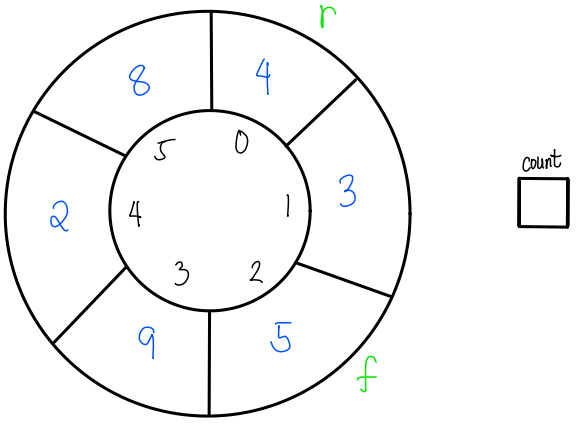
ex1 size: 6 data: 4 ex2 size: 6 data: 3

$$6 \overline{)4}\ 0 \quad R4 \qquad 6 \overline{)3}\ 0 \quad R3$$

↳ Use modullos every time you use the rear

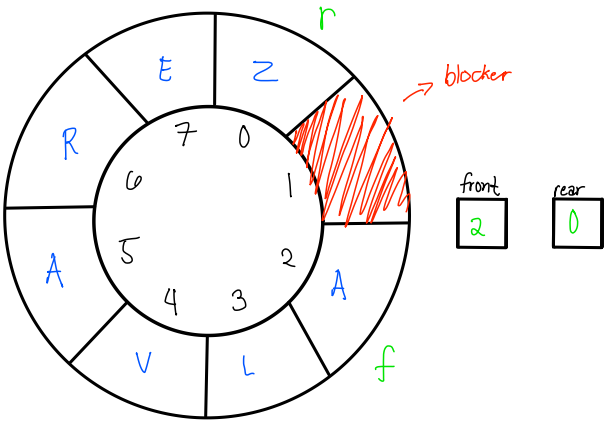
(6)
(rear + 1) % MAX == 0
enq(4);





↳ Another example with size 8.
Now let's insert A, L, V, A, R, E, Z.

enq(A)
enq(L)
enq(V)
enq(A)
enq(R)
enq(E)
enq(Z)



We know that index 0 is the rear, which is the last index.

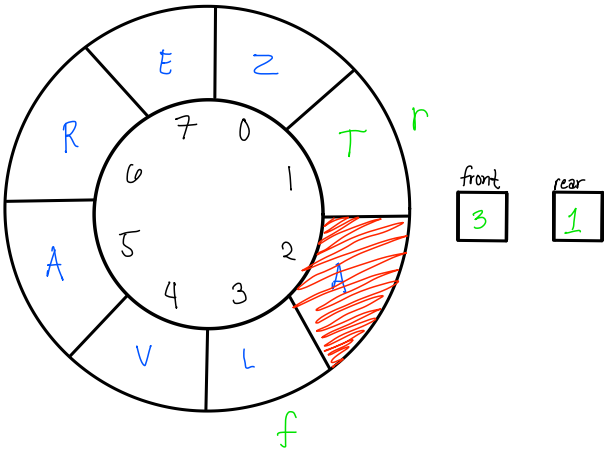
- Question: How would you know that there is only one element in the array?
  Answer: When your Front is equal to Rear, there is only one element. front == rear ⇒ 1 element

  Question: How would you know that there is an empty list?
  Answer: If Rear is in front of Front, then it is empty. front == r + 1 ⇒ Empty

  Question: How would you know that the array is full?
  Answer: If your Rear is next to the blocker, it's full. front == r + 2 ⇒ Full
  ↳ an indicator that the array is full.

- Now we wanna insert letter T.

enq(T)



Note:
- In terms of code, you don't have the blocker, it's just a concept.
- Every time you dequeue() the blocker will follow.

ASSIGNMENT

```
typedef struct {
    String name;
    String specie;
    String color;
    int breed;
    int age;
} Bird;
```

```
typedef struct {
    Bird breedBirds[10];
    int start
    int end;
} Queue;
```

Similar to the last Assignment just do it in Queue.

1.) void enqueue; } can access index (?)
    void dequeue;
    Bird front;

2.) void enqueueUnique; (color) } operations only

3.) int countQueue;

Sept. 14, 2026

ASSIGNMENT — ANSWER

```
typedef struct {
    String name;
    String specie;
    String color;
    int breadi;
    int age;
} Bird;
```

Similar to the last Assignment just do it in Queue.

1.) void enqueue;
    void dequeue;
    Bird front;
    } can access index (?) } these are the operations

2.) void enqueueUnique; (color) } operations only

3.) int countQueue;

```
typedef struct {
    Bird breadBirds[MAX];
    int start    // front
    int end;     // rear
} Queue;
```

ANSWER

```
void enqueueUnique ( Queue *Q, Bird B) {
    if( Q->start == (Q->end + 2) % MAX) {      // error checking
        printf("List is Full");                 // full
    } else {
        temp = front (*Q);
        while (Q->start == ((Q->end+1) % MAX)) {   // process   // empty
            isDuplicate = (strcmp (temp.color, B.color)) 0: 1);
            dequeue (Q);
            enqueue (&temp Q, temp);
            temp = front (*Q);
        }

        *Q = temp Q;
        if( isDuplicate = 1) {
            printf ("Duplicate found");
        } else {
            enqueue (Q, B);
        }
```

O(n+n) ⇒ has 2 loops
O(n) ⇒ only has 1 loop.

Note:

- Overall, Stack needs to be pop() and push().
- Queue, you can just put it in a temp and copy it to the original.

queue:

*Q	temp
X B C D	A B C D

CURSOR-BASED

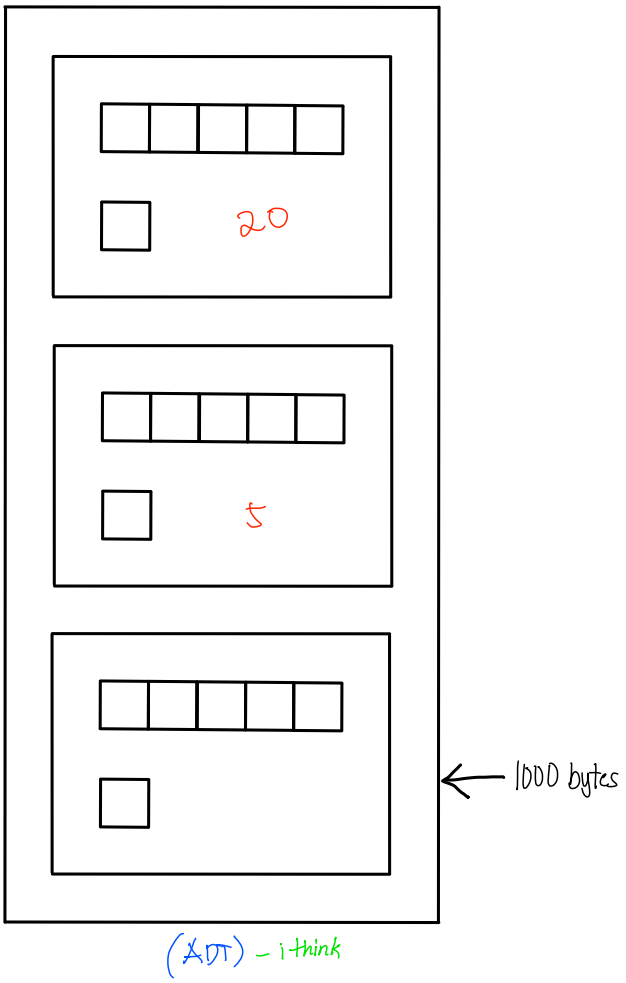
STRUCTURES

Reference the following structures, answer the following prompts with two variations: firstly, access the data without a pointer and secondly access with a pointer.

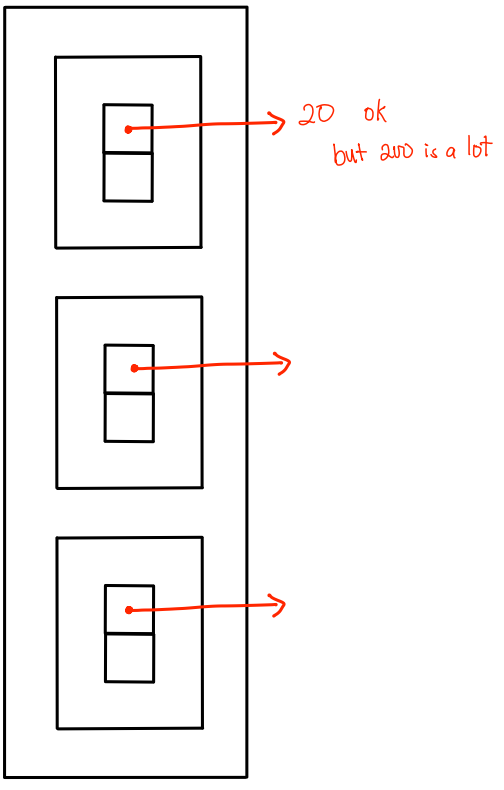
```
#define MAX 30;
#define char String[50];

typedef struct{
  String bookName;
  String authorName;
  String publishingHouse;
  int publishingYear;
  int totalPageNum;
}Book;
```

```
typedef struct{
  Book books[20];
  int bookCount;
}Shelf;

typedef struct{
  Shelf shelves[3];
  int totalBookCount;
}BookShelf;
```

```
BookShelf BS;
BookShelf *BointR;
```



(ADT) - i think



- Linked List is more dynamic.

Now, we want something that is time-efficient and space-efficient. → Cursor based

↳ causes O(1) instead of O(n).

Example of a Cursor-Based

↱ Young Adult

YA 0, 3

FA 1, 2

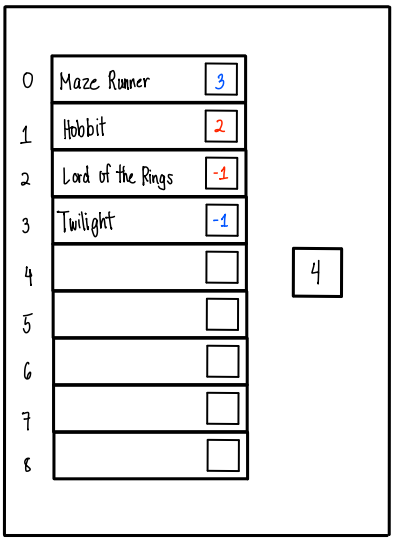
↳ Fantasy

YA

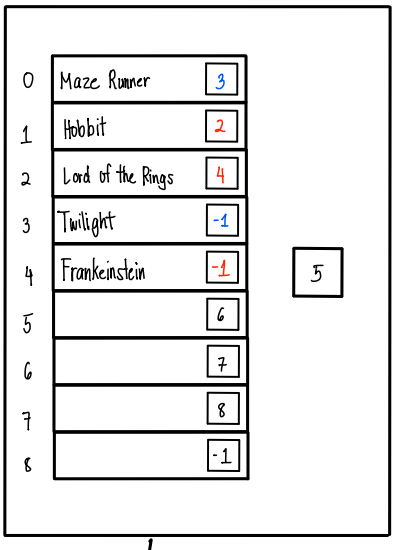
Fantasy

Fantasy

YA



Now I want to add Frankeinstein.



YA 0, 3

FA 1, 2, 4

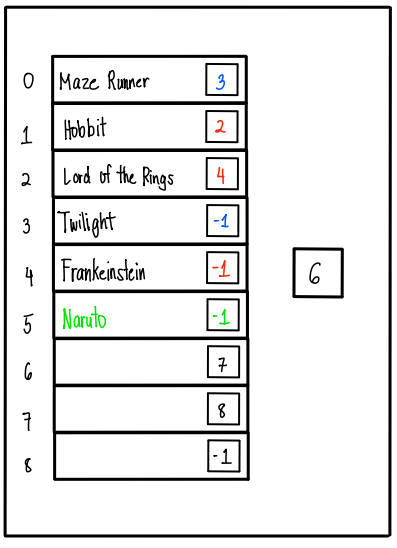
MA

Adding "Naruto" manga

YA 0, 3

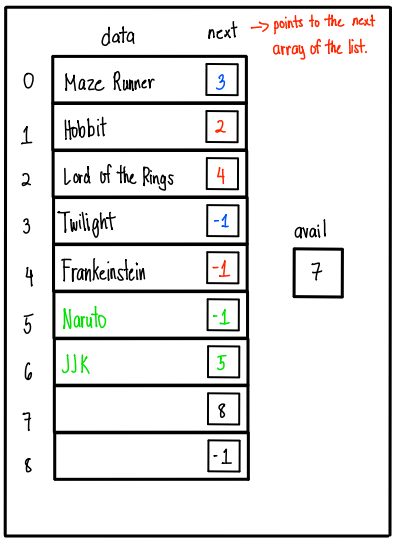
FA 1, 2, 4

MA



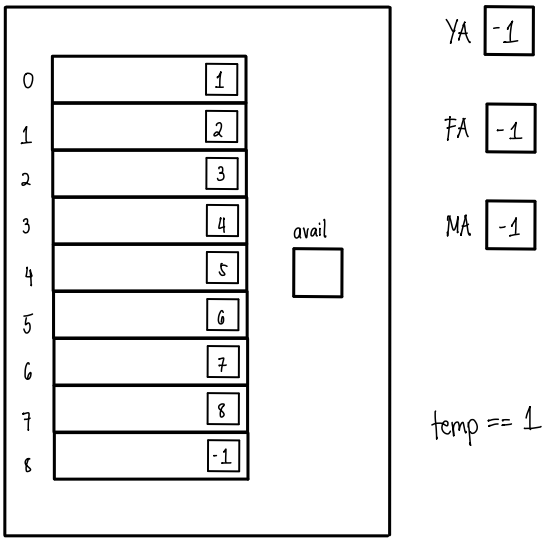
Now I want to add "JJK" and make it first.

YA 0 FA 1 MA 6

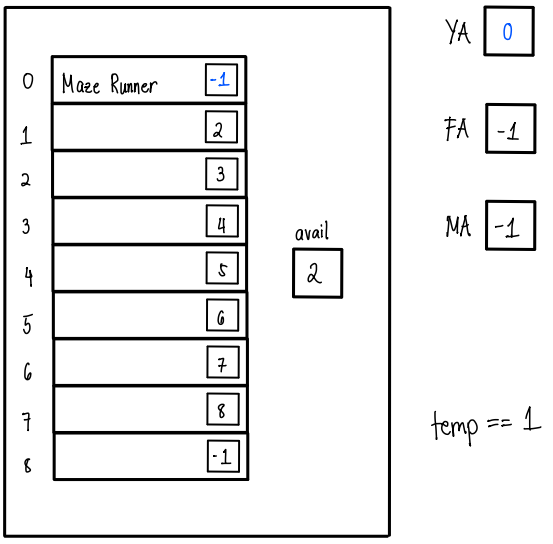


Now I'll do it again but in detail

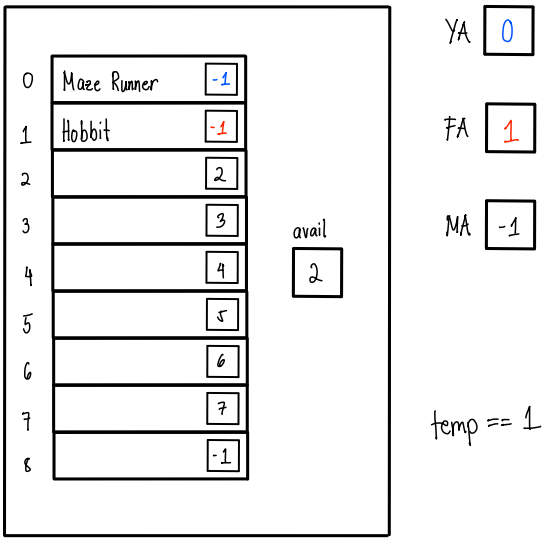
1



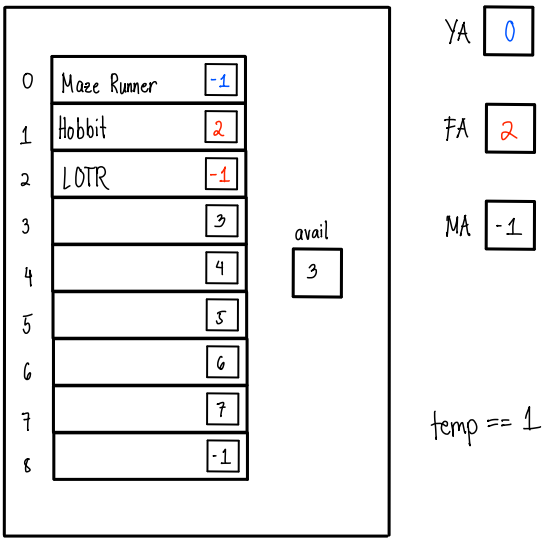
2



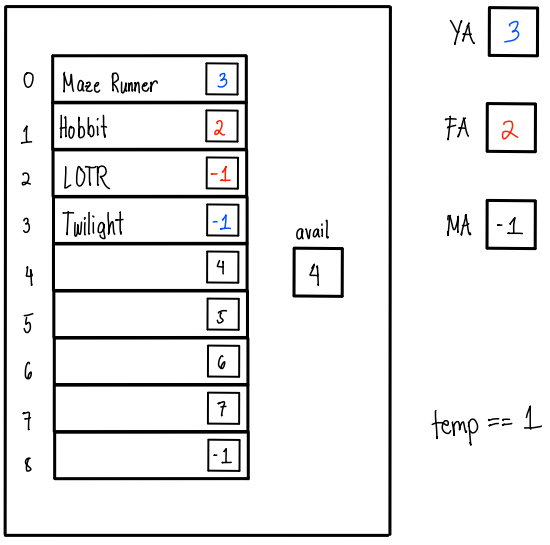
3



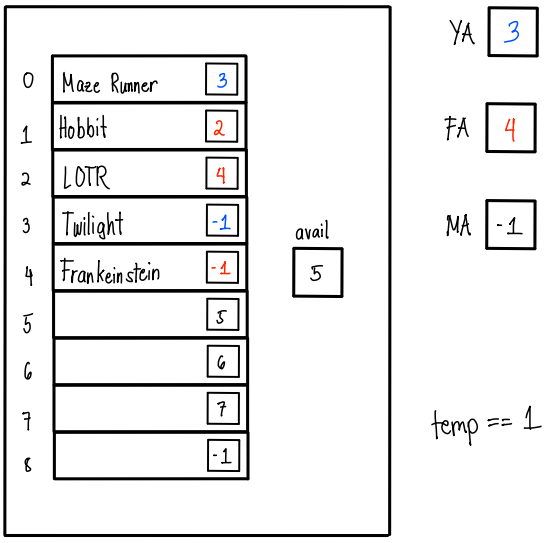
4



5

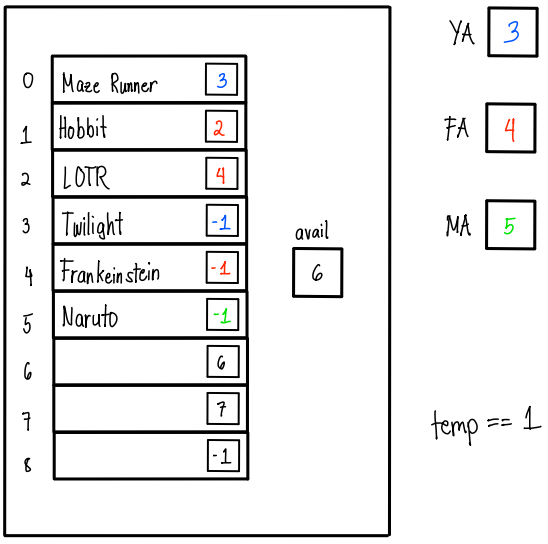


6

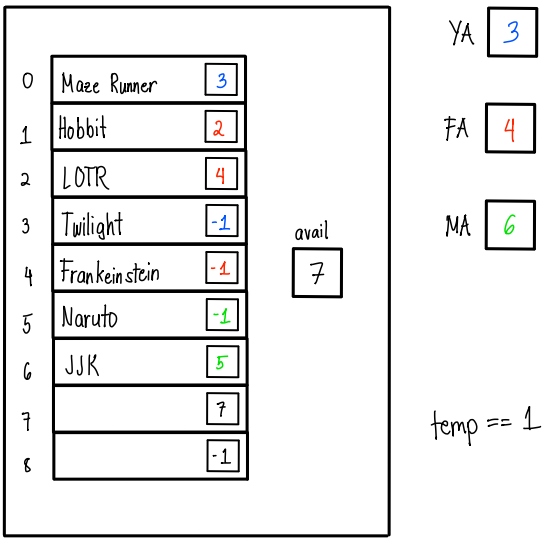


(we want JJK to be on the top)

7

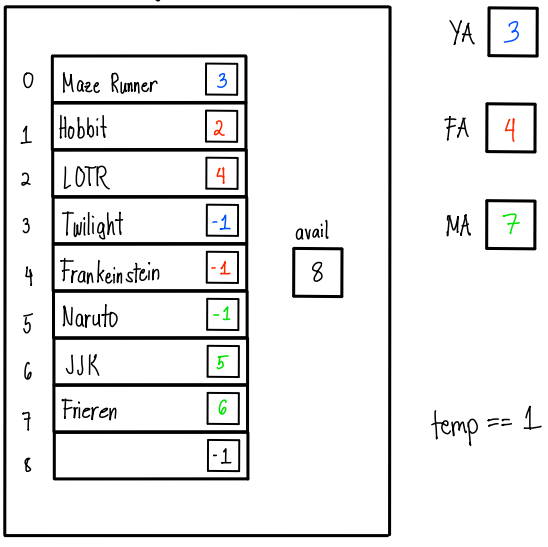


8



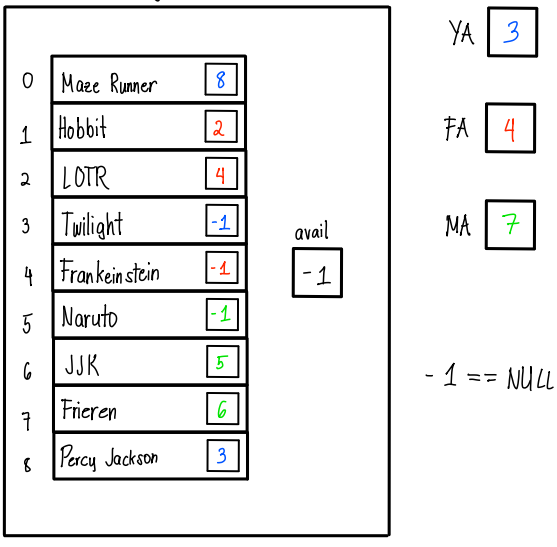
Adding new data

9



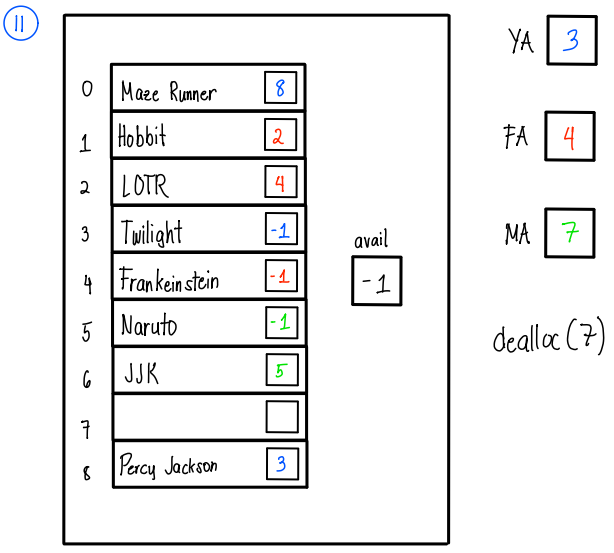
Adding new data

10

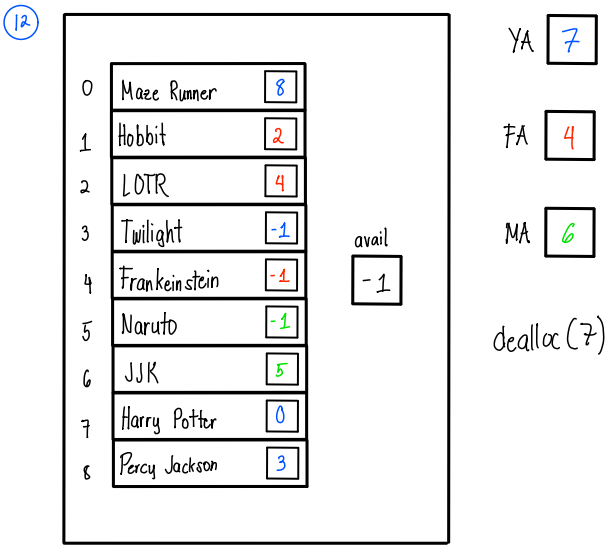


YA	MR -> PJ -> TW
FA	H -> LOTR -> F
MA	Frier -> JJK -> N

Now, we want to delete Frieren.



Adding "Harry Potter"



Note: • Use double pointers for Cursor - Based.
• You can use next & prev but they are not efficient.

ASSIGNMENT 3

STRUCTURES
BookShelf
-> Not required but pwede pa check (which ni ingon nas maam na ikeep ra ang paper).

ANNOUNCEMENT:

Exam: on Saturday Sept. 19, 2026
There are 4 parts:
1.) ADT List 3.) Queues
2.) Stacks 4.) Cursor - Based (BONUS)
↳ more useful in other implementation throughout the whole semester.

[E1] - Music Playlist - Cursor Based by Edrian Guanzon

(Sept. 16, 2026)
• On Wednesday naay coding activity related sa exam (Stacks, Queues, List & CB)

09285767290
Call me <3

Sept. 16, 2026

REVIEW FOR EXAM

GRADING

CO2 - 12%

Conceptual Topics

- ADT List
- Stacks
- Queues
- Big O Notation

CO1 - 18%

Practical Coding

- ADT List
- Stacks
- Queues
- Cursor Based (BONUS)
  - all or nothing
  - it's 1.0 or 5.0 (Basically if you don't get All the test cases you'll have 5.0 but if you do get All test cases you'll have 1.0)

1.0 → all test cases → eff
1.5 → all test cases → inefficient
2.0 → 90% ~ 99%
2.5 → 60% ~ 90%
3.0 → 60%
3.5 → 10% ~ 60%
4.0 → 1 test case / 0 test case
5.0 → ABSENT

The total number of test cases and 60% of that, you'll pass.

STACK - pop(), push(), etc.
QUEUE - enqueue(), dequeue(), etc.

EXAMPLE OF THE EXAM

1.) Which of the following is the most appropriate description of stack?

A: Data that is stored first is retrieved first. → retrieved last (?)

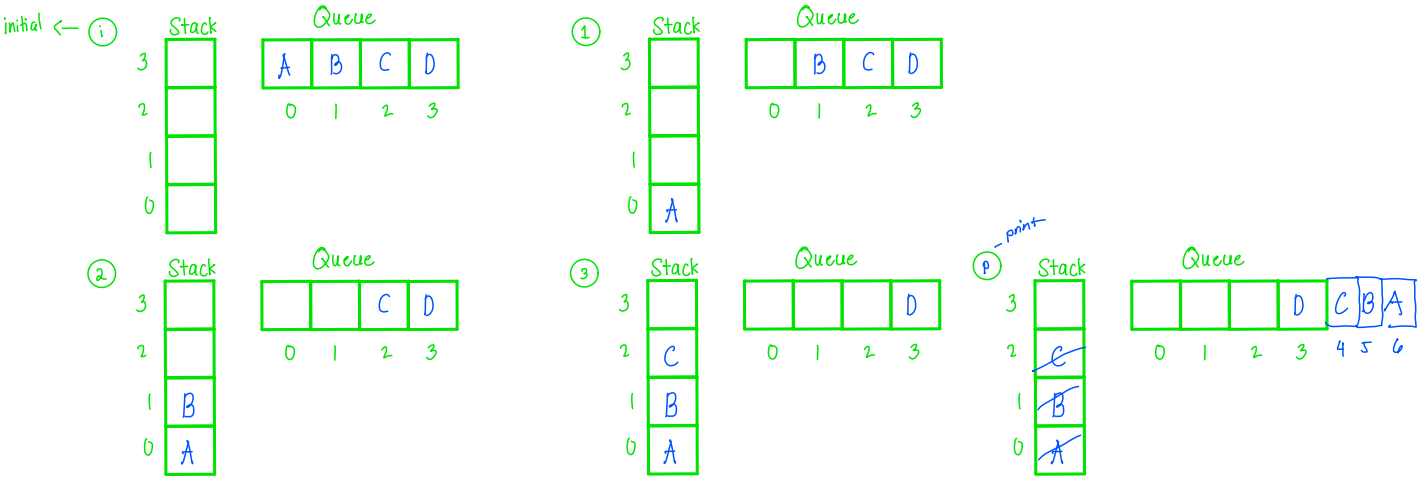
For Queue: Data that is stored last retrieved last.

2.) For a pair of queue and a stack, two operations "pop_enq" and "deq_push" are defined as follows:

pop_enq: Pops an item from stack and puts it into the queue.
deq_push: takes an item from the queue and pushes it into the stack.

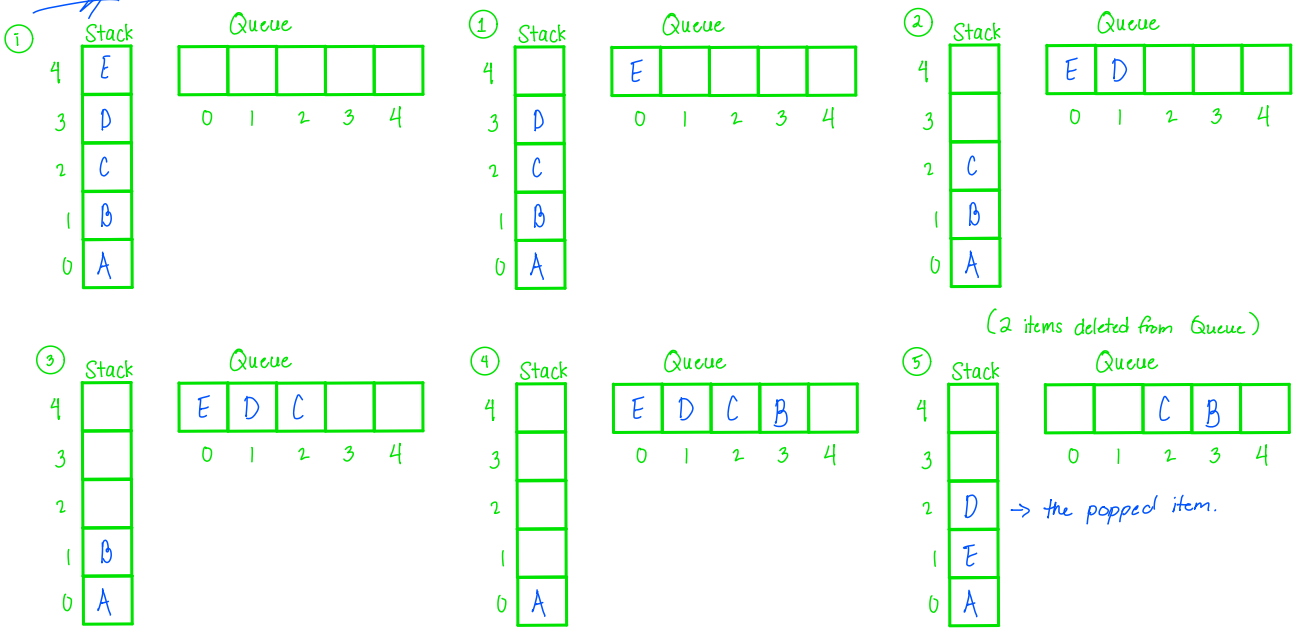
When four items A, B, C, and D are stored in the queue, what is the minimum number of deq_push operations required in order to reverse the order of the four items by using only these two operations?

A: 3



3.) The five items: A, B, C, D, and E are pushed in a stack, one after other starting from A. The stack is popped Four items (or times?) and each element is inserted in queue. The two elements are deleted from the queue and pushed back on the stack. Now one item is popped from the stack. What is the popped item?

A: D



4.) Which of these statements about array list is false?

A: The number of elements in the array list has no limit.

- Array will always have number of bytes.
- Array index is sequential.

5.) What is the time complexity of inserting N elements into an initially empty sorted linked list?

A: O(N)

- O(N log N) -> this can usually be used in trees.

6.) Given the code snippet below, you must identify the error.

```
// Stack definition
typedef struct {
    int items[MAX];
    int top;
} Stack;

// function prototype
void initializeStack(Stack *s);

// code snippet
Stack warehouse[NUM_STACKS];
for (int i=0; i<NUM_STACKS; i++) {
    initializeStack(warehouse[i]);
}
```

A: initializeStack(warehouse[i]);

it should be "initializeStack(&warehouse[i]);"

7.) Which of the following operations can be performed in O(1) time when ADT list is represented in memory using linked list for an array implementation?

A: B & C

B. Insertion of the element at the front of the list implemented using linked list.

C. Deletion of the last element of the list implemented using an array implementation.

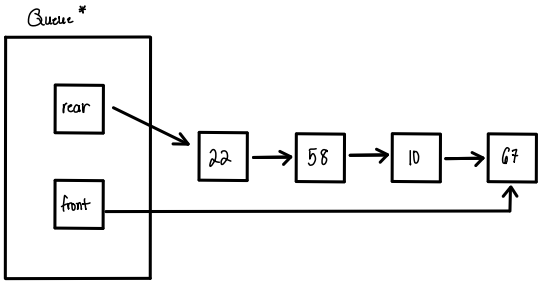
8.) In the array and linked list implementation of list, which of the following is true?

A: The time complexity of deleting the element at the last position is O(1) for array and O(N) for linked list.

9.) Execution time of an operation is said to be constant if the time it takes to complete the operation is the same when the array list is empty or full. Which of the follow operations can be performed in the constant time?

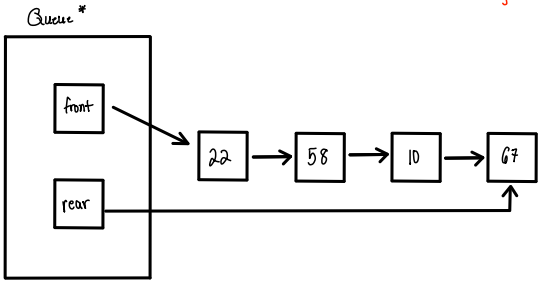
A: Insert at the last position in the array list.

10.) In the illustration of the Queue below, which of the following statements is/are true?



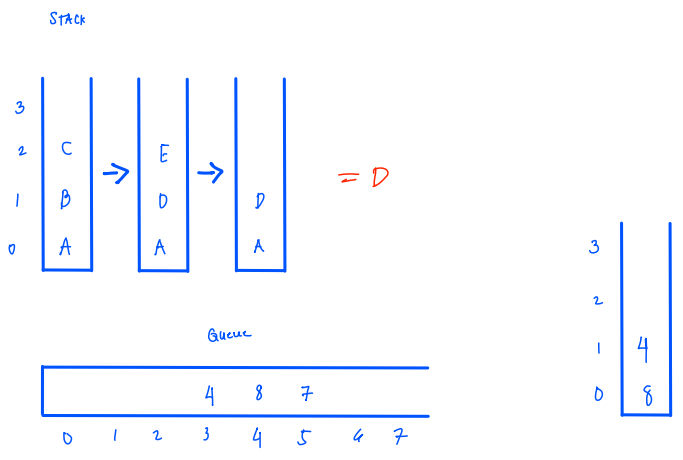
A: Time complexity of Enqueue is O(1) and Dequeue is O(1).
↳ O(n)
says ChatGPT and Gemini

If Reverse:



A: Time complexity of Enqueue is O(1) and Dequeue is O(1)

CODECHUM ACTIVITY PRACTICE (unfortunately there's no instructions since it was from a previous Exam)



Sept. 21, 2026

Note: It's in Visual Studio Code. Will upload the files in GitHub.

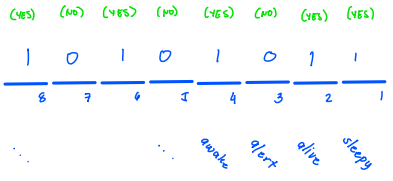
Sept. 23, 2026

SETS

```
String[50];
typedef struct {
    String name;    // 50
    int id;         // 4
    int gender;     // 4
    int uscStud;    // 4
} Student           = 62

typedef struct {
    Student id;     = 62
    int survey[20]; = 20 x 4 = 80
    int trewFols[20]; = 20 x 4 = 80
} Questionnaire;    = 222
```

Q: In this structure definition, how many bytes is Questionnaire? 222

```
char x;  // 1 byte
   ↳ 8 bits
```



Now how many bytes if we have:

Questionnaire Tests[30];

96 * 30 = 2,880

```
  1
x 96
  30
  00
 288
2880
```

```
short int;  // 2 bytes
   ↳ 16 bits

int; // 4 bytes
   ↳ 32 bits
```

To make it more efficient:

```
String[50];
typedef struct {
    String name;    // 50
    int id;         // 4
    char gender;    // 1
    bool uscStud;   // 1
} Student           = 56

typedef struct {
    Student id;     = 56
    int ~~char~~ survey[~~20~~]; = 4
    int ~~char~~ trewFols[~~20~~]; = 4
} Questionnaire;    = 64
```

Q: In this structure definition, how many bytes is Questionnaire? 64

Now how many bytes if we have:

Questionnaire Tests[30];

1) 96 * 30 = 2,880

2) 64 * 30 = 1,920

```
  64
  30
  00
 192
1920
```

Sept. 23, 2026

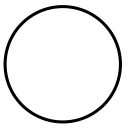
Bits

Note: In '01code.c' file:

```
int i = i * (sizeof(value) * 8 - 1);
```

↳ 8 here means there are 8 bits.

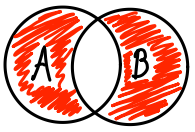
note: I honestly don't remember asa ni ↑



Definition

- **Sets** – are unique collection of elements

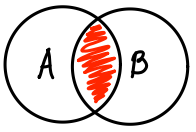
Union



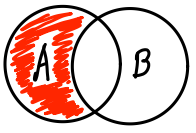
Merge ↑

(AB) ∩ (B-A)

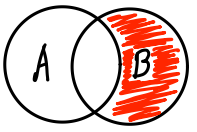
Intersection



Difference



A - B



B - A

Sept. 28, 2026

Continuation On Sets

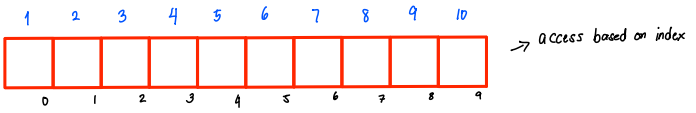
10 Questions Y/N:

int Array[10];

to shorter it, it should be:

Char / bool A[10]; =

Yes/No, True/False → Char; Y/N → Char; bool → T/F (1/0)



→ access based on index

int B

to shorten it, it should be:

short int B = { _____ } _____

2^{15} 2^{14} 2^{13} 2^{12} 2^{11} 2^{10} 2^{9} 2^{8} 2^{7} 2^{6}

1 byte == 8 bits

2 byte == 16 bits

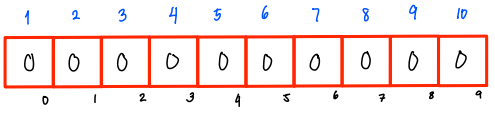
4 byte == 32 bits

i = sizeof(B) * 8 - 1; → start at the most left side

↳ shift based on the size of the array (?)

To initialize everything to 0 we use calloc()

Char / bool A[10]; =

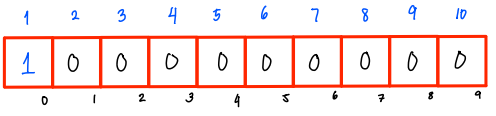


short int B = { 0 0 0 0 0 0 0 0 0 0 } _____

2^{15} 2^{14} 2^{13} 2^{12} 2^{11} 2^{10} 2^{9} 2^{8} 2^{7} 2^{6}

How to make the 1st Question (Q) to 1?

Char / bool A[10]; =



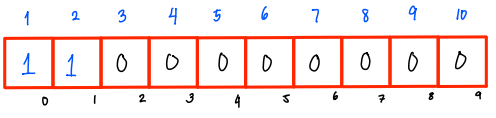
A[0] = A[0] | 1;

short int B = { 1 0 0 0 0 0 0 0 0 0 } _____

2^{15} 2^{14} 2^{13} 2^{12} 2^{11} 2^{10} 2^{9} 2^{8} 2^{7} 2^{6}

Given this sample:

Char / bool A[10]; =



A[0] = A[0] | 1; A[1] = A[1] | 1;

(i >> 4) ... OR

B = B | (i >> 4); B = B | (i >> 4);

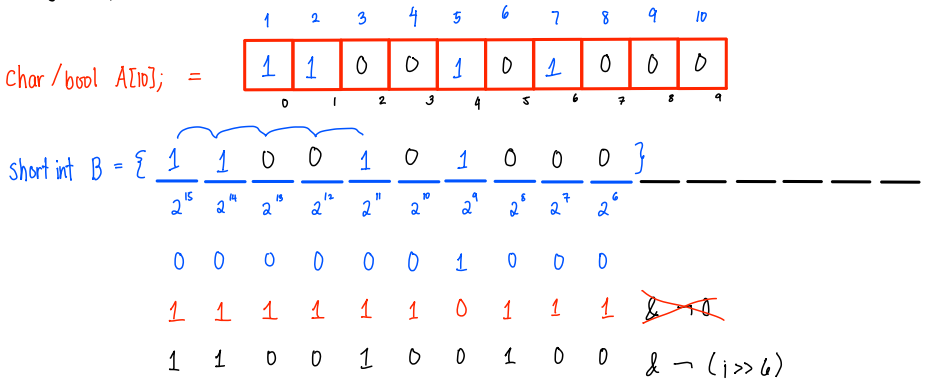
short int B = { 1 1 0 0 0 0 0 0 0 0 } _____

2^{15} 2^{14} 2^{13} 2^{12} 2^{11} 2^{10} 2^{9} 2^{8} 2^{7} 2^{6}

- 8 bits (minus 8 bits)

B = 00 x 10

Now adding new questions:



note: if you start from the left, shift to the right.
if you start from the right, shift to the left.

sizeof() => is for dynamic

Now we want to delete the 7th Question:

0 0 0 0 0 0 1 0 0 0

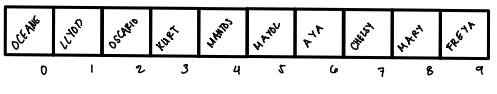
symbol:	mean:
~	-> inverse

- deletion means it won't exist in the array list.
  - ↳ difference => & (AND)
- insert
  - ↳ union => | (OR)
- displayBitPattern
  - ↳ intersection =>

Ex. 1

A = { 1, 3, 5, 6, 8 }

How to represent in an array format?



-> difference of B

A = { 1, 2, 3, 5 }	-> all male	C & ~B =>	all females that are not sit ins { 0, 4, 6, 7, 8 }
B = { 9 }	-> sit in	B & ~C =>	all sit in given they are all females { 9 }
C = { 0, 4, 6, 7, 8, 9 }	-> all female	A & D =>	who is the .. I think { 3 }
D = { 3, 4, 8 }	-> 1st row		male in the 1st row ?

Sept. 30, 2026

DICTIONARY

(A) Example 1

1.) Mario
2.) Peach
3.) Luigi
4.) Toad
5.) Pauline
6.) Daisy
7.) Bowser
8.) Wario
9.) Yoshi
10.) Waluigi

Set A = {1, 4, 5} == red
Set B = {2, 5, 6} == females
Set C = {1, 3, 8, 10} == wearing overalls
Set D = {4, 7, 9} == monsters

(B) Definition

- Dictionary let's you search immediately without having to traverse from the beginning of the list.
- This is the fastest Data Structure.

(C) Let's say we have this

Red	Green	Blue	Yellow
shirt	id	wires	Bowser
			hoodie

→

	hoodie
	shirt
	id
	wires

if we were to put it in a list

Can you tell what color they are? No

0	hoodie
1	shirt
2	id
3	wires

That's why we have index for list/array.

Red 0	Green 1	Blue 2	Yellow 3
shirt	id	wires	Bowser
		daisy	hoodie

→

	hoodie
	shirt
	id
	wires

Can you tell what color they are? No

0	shirt
1	id
2	wires
3	hoodie

That's why we have index for list/array.

Q: Which is easily identifiable? The word

Hashing - determines the location of the element.

Red 0	Green 1	Blue 2	Yellow 3
shirt	id	wires	Bowser
mario		daisy	hoodie
pencil case			earrings
fan			

→

Context: By using your eyes you can easily identify that:

Hash (earrings) → 3 ↵ } is at index 3.

Hash (fan) → 0 ↵ } is at index 0

7 6 5 4 3 2 1 0
128 64 32 16 8 2 1 0

(D) Hashing example — in terms of 10th place xxx

Hash (568) → Hash (568)
↳ returns 8 ↳ returns 6

Hash (679) → Hash (679)
↳ returns 9 ↳ returns 7

Hash Context

Try this out: ACTIVITY

1.) Hash (568) == 5
Hash (679) == 6

2.) Hash ("Alegado") == 0
Hash ("Dominic") == 3

3.) Hash (3,5,7) == depends
miss ex. Hash (3+5+7) == 15

CODE:

1.) a.)

b.) return (num % 1000) / 100; // still correct

c.) return (num / 100) % 10;

2.) a.) return x[0] - 65; ← using ASCII if (s[0] < 'A' or > 'a') ? _ : _

b.) return name[0] - 'A';

c.) return toupper(last[0]) - 'A';

3.) a.) return ((r+g+b) < 65) ? r+g+b : 0; ← some

b.) return ((r*1) + (g*2) + (b*3)) % 64; ← most

c.) return (r/64*16 + g/64*16 + b/64*16) % 64; ← most

① OR

b.) return ((r*1) + (g*2) + (b*3)) % 64;

c.) 9 4 9 == 22
17 2 3 => 22

② OR

b.) return ((r*7) + (g*13) + (b*11)) % 64;

c.) 9 2 3 == 63 + 26 + 33 % 64 = 58
17 1 1 => 119 + 13 + 11 % 64 = 15

OPEN HASHING

$10 \overline{)72}$ → 7, −70, r. 2 ; $10 \overline{)6}$ → 0, −0, r. 6 ; $10 \overline{)91}$ → 9, −90, r. 1

Quotient / Divisor | Dividend / Remainder

Dividend ÷ Divisor = Quotient

20 ÷ 10 = 2

47 ÷ 10 = 4

Given : Set A = { 20, 47, 22, 5, 65, 72, 6, 91 }

Since there's no 21 we will not insert

$10 \overline{)20}$ → 2, −20, 0 (index)

$10 \overline{)47}$ → 4, −40, r. 7 (index)

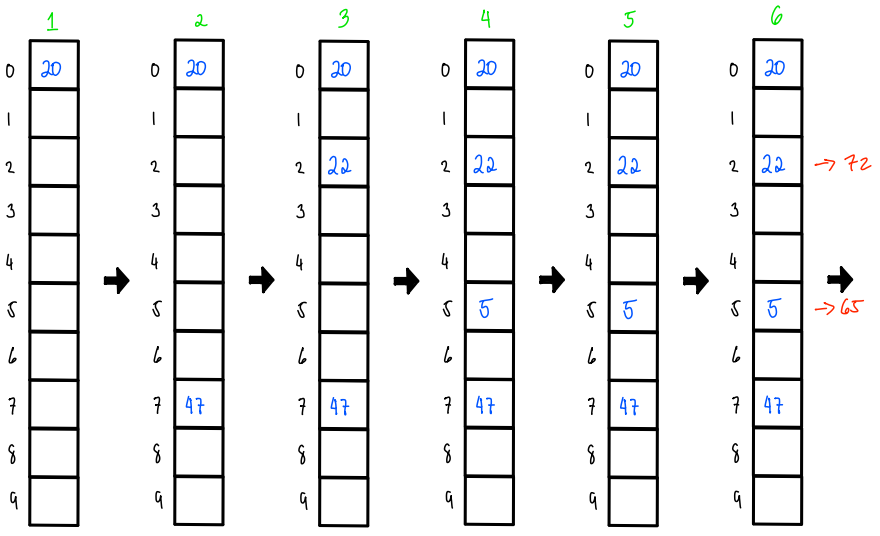
$10 \overline{)22}$ → 2, −20, r. 2

$10 \overline{)5}$ → 0, −0, r. 5

$10 \overline{)21}$ → 2, −20, r. 1 — hash(21)

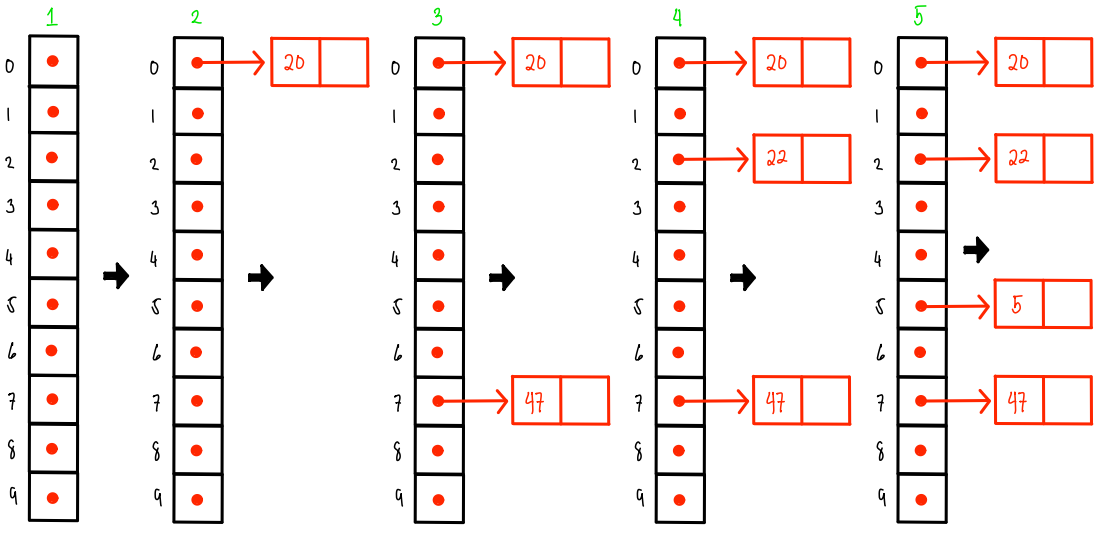
$10 \overline{)65}$ → 6, −60, r. 5 — hash(65)

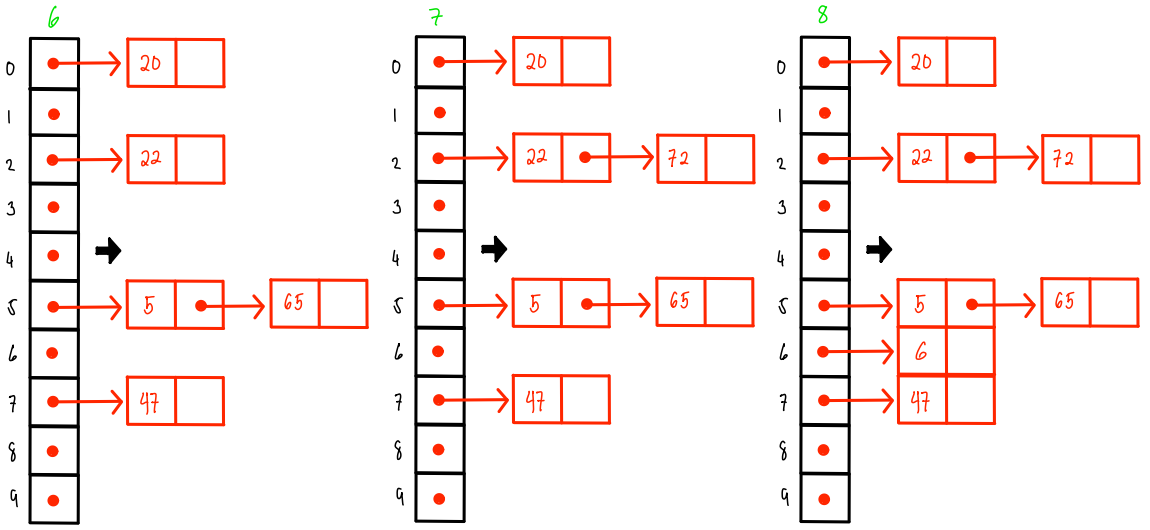
$10 \overline{)72}$ → 7, −70, r. 2

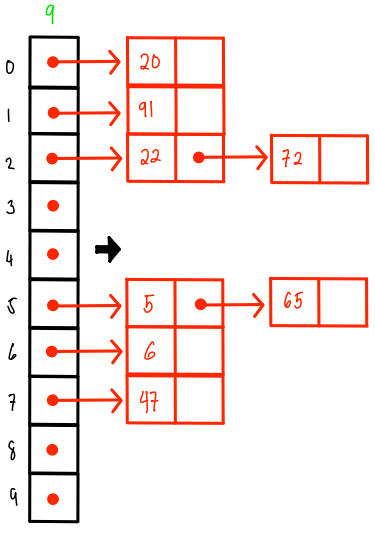


The concept is like this in array but it will get overwrite. The value of index 2 is 22, it will change to 72 and we don't want that. So we use array of linked list.

We use an array of linked list.

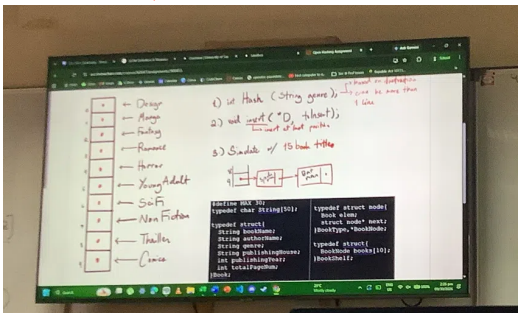






ASSIGNMENT

1.) Canva → On Paper [BookShelf]



2.) CPRC 5 (C Programming Reading Comprehension)

3.) CodeChum & Progma

If you want to meet next week on midterm exam then talk about it as a class. Nagy class if everyone agrees.

Im done I think. I'll check my phone if I miss something.
(Oct. 10, 2026)

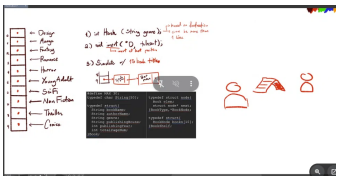
Start rewrite here

Oct. 14, 2026

- ASSIGNMENT (OPEN Hashing)
- Closed Hashing
- Priority Queue

Oct. 26, 28 & Nov. 4 = Review for Exam

ASSIGNMENT



3 WAYS TO DO HASHING

1.) if (strcmp(genre, "Design"))
    retVal = 0;

2.) arr[10] = {"Design", "Manga"..} → a search for loop
    for (strcmp(1) { }
        return Idx;

3.) switch (genre[0])
    case 'D' retVal = 0;
    case 'M' retVal = 1;
    case 'F' = 2;

these 2 is not the most ideal/efficient hashing

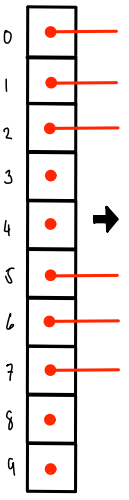
is most efficient

- Don't use multiple return statements

void insert ()
    insert()
    error { int hashIdx = Hash (genre)
        if hashIdx > 9) { error "}

process { **trav = D->books[hashIdx];

update {



OPEN HASHING

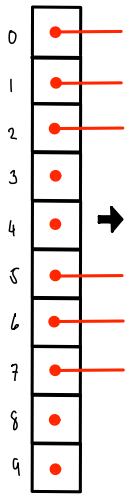
Q: What will happen if we cannot use pointers in Hashing?

hash(20)

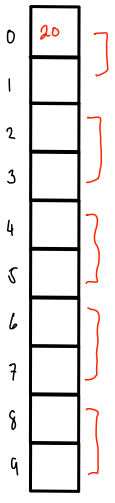
$10 \overline{)20}$ → 2, remainder 0 → index

$10 \overline{)47}$ → 4, remainder 7

This part can be found here ♥ above



What if our hash() generates a range?



hash → (0, 2, 4, 4, 5)

0, 1, 2, 3, 4 — index

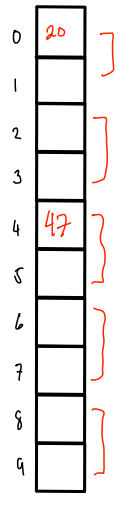
(x % 5) * 2

SETA =

{20, 47, 22, 5, 65, 72, 6, 91}

20 % 5

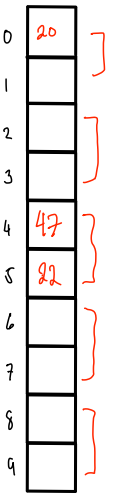
$5 \overline{)20}$ → 4, remainder 0

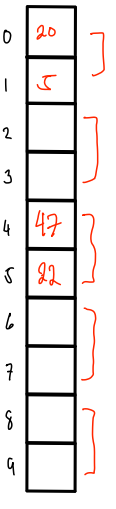


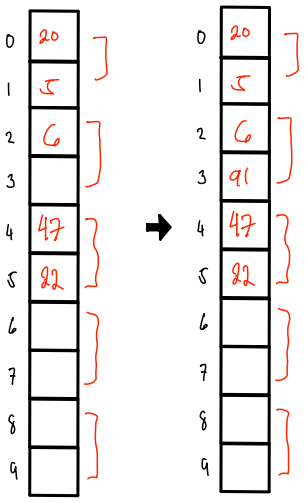
47 % 5

$5 \overline{)47}$ → 9, remainder 2

$2 \times 2 = 4$



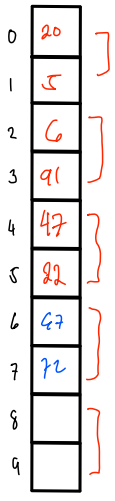




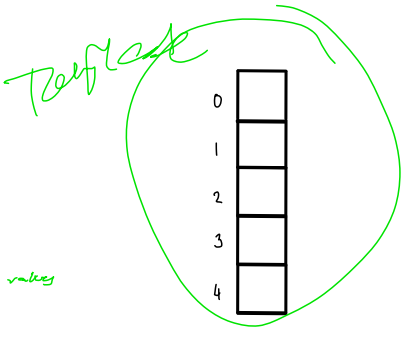
CUCKOO HASHING
↳ applying linear hashing

BUCKET - using of multiple spaces
- you are limited in your space use

if we add

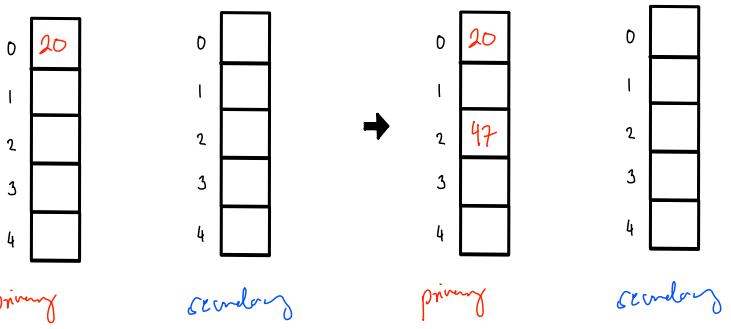


it will cause a displacement.



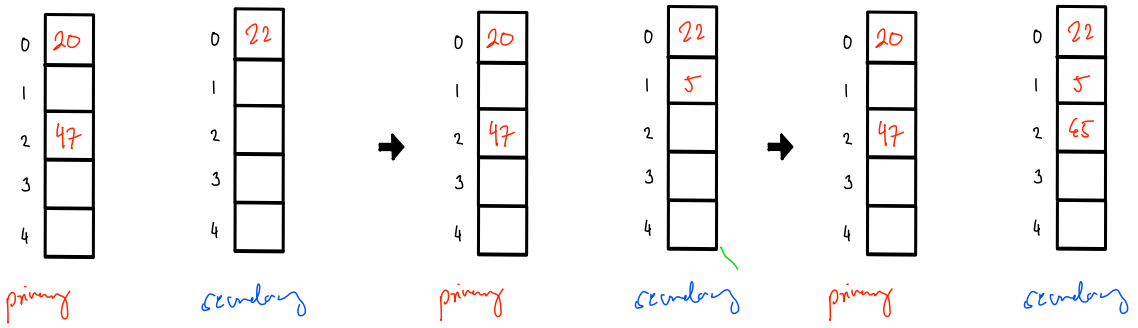
SETA =
{20, 47, 22, 5, 65, 72, 6, 91} (x % 5)
↳ since there are 5 values

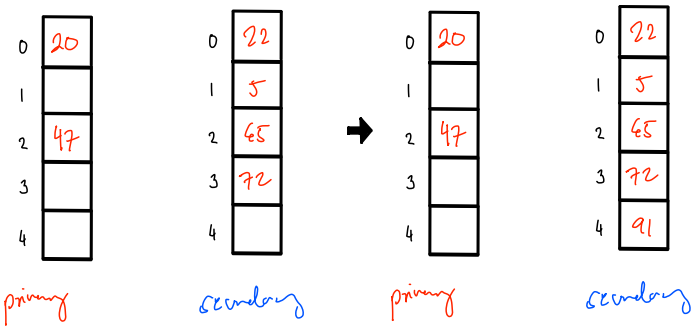
What if we take an array and split it into two?



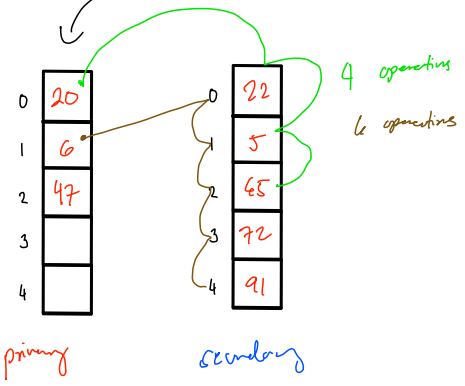
$$5 \overline{)47} \quad 9 \quad -45 \quad 2$$

$$5 \overline{)22} \quad 4 \quad -20 \quad 2$$

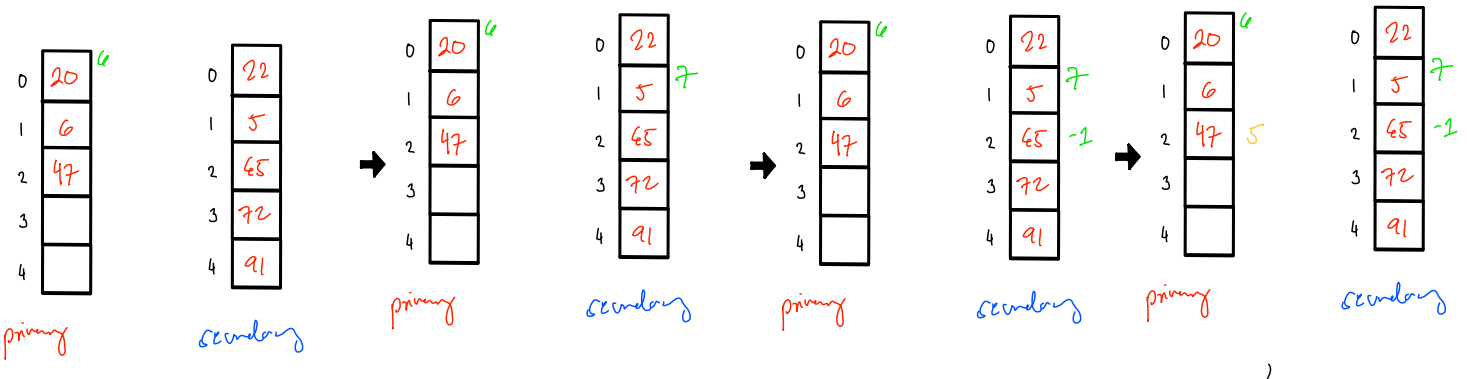


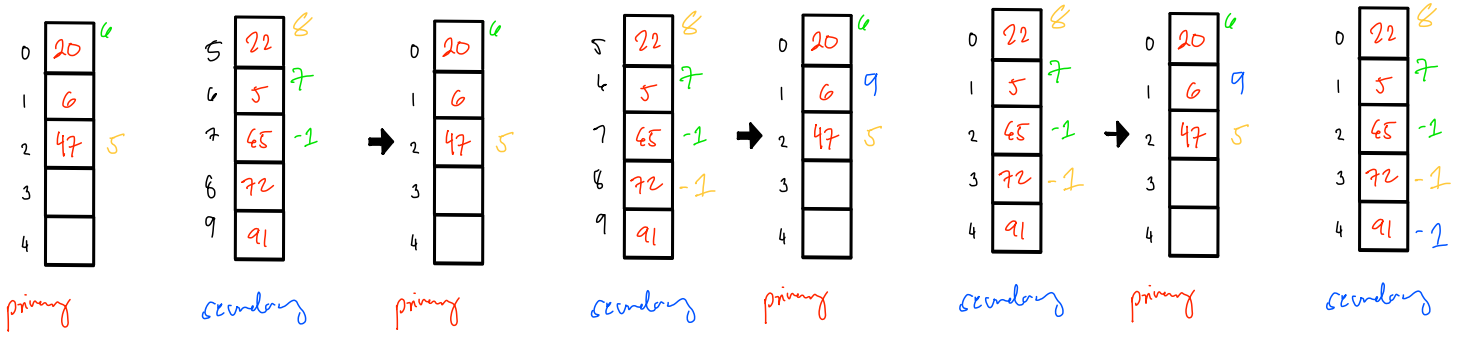


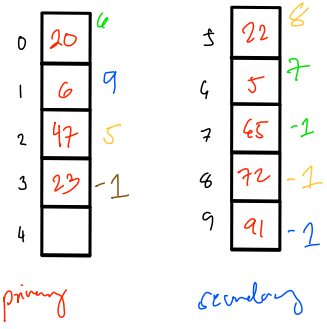
This is closed Hash but this is a problem. i.) It is not ordered correctly.



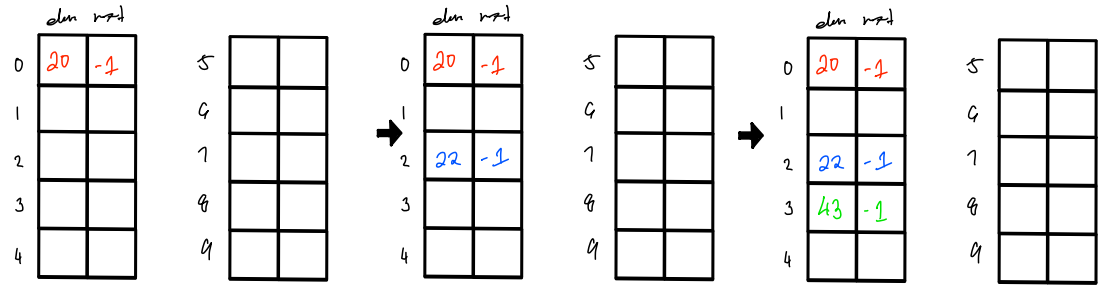
What if we link in terms of indexes? We cannot use pointers.

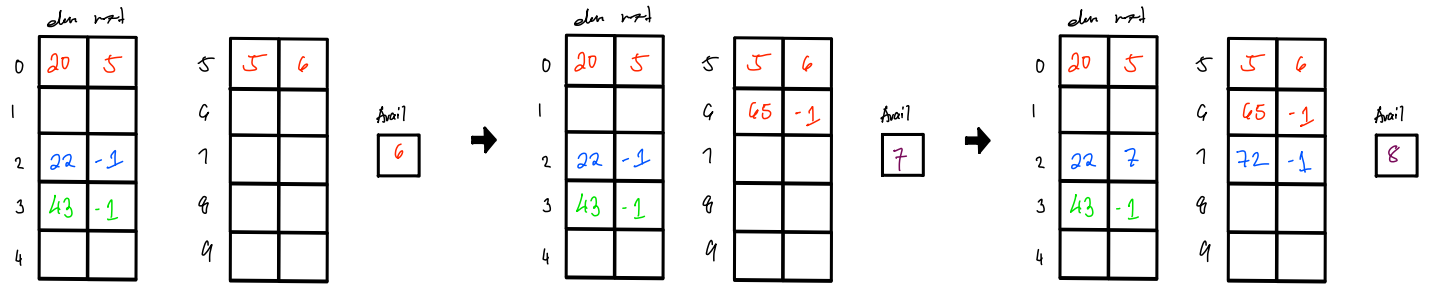


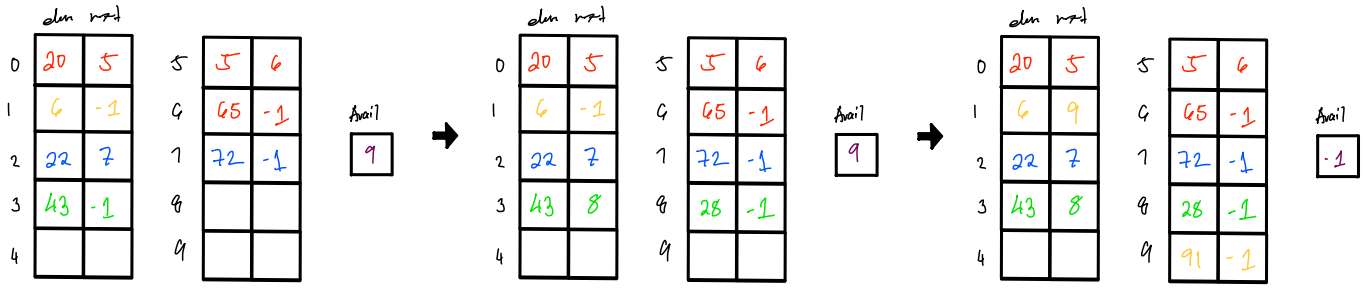




Instead of using buckets we can use Progressive Overflow

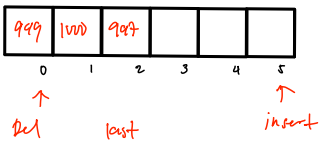






Q: Is it possible to get the perfect hash?
Yes but not true.

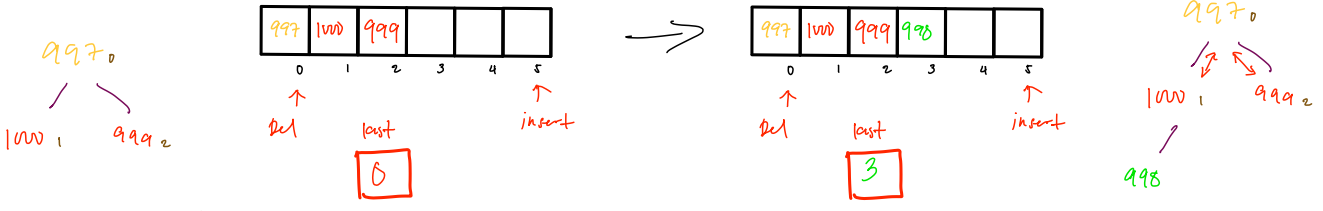
PRIORITY QUEUE



0

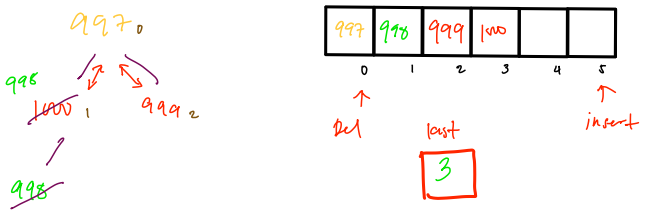
Binary heap
- to take it two at a time
- a tree's implementation
- it goes first

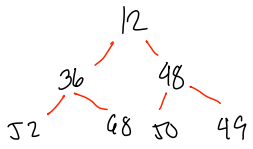
L most is lesser than the RIGHT most.

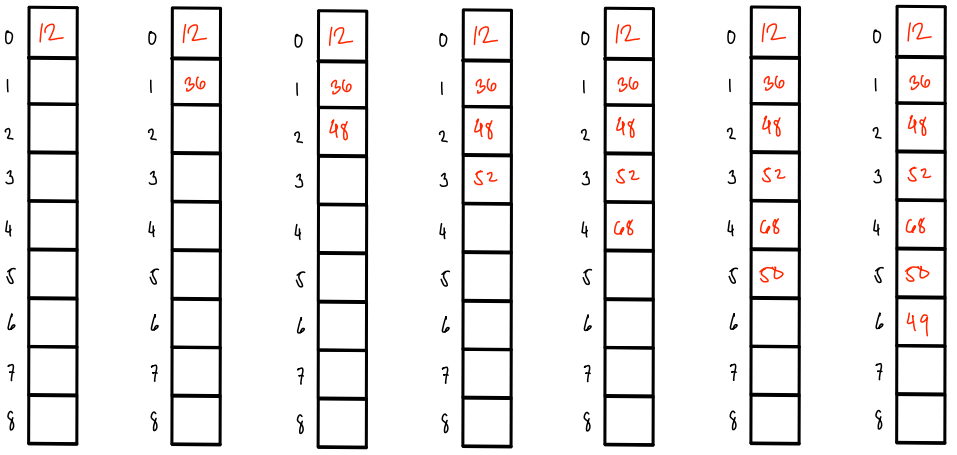


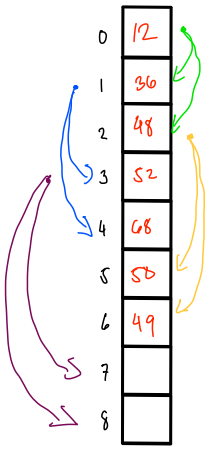
P < C
- Parent node is less than the child node.
- the sibling priority num. don't matter, just the parent & child.

Min Heap









index

0 → 1, 2

1 → 3, 4

2 → 5, 6

Parent n → child

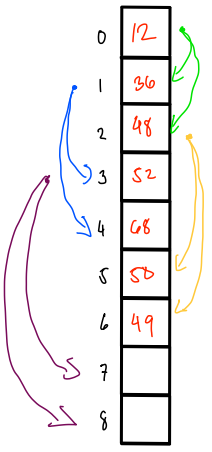
LC (2n + 1)

RC (2n + 2)

ex.

n = 3

LC: 7 / RC: 8



index

0 → 1, 2

1 → 3, 4

2 → 5, 6

Parent n → child

LC (2n + 1)

RC (2n + 2)

ex.

n = 3

LC: 7 / RC: 8

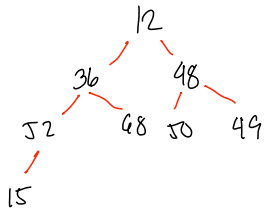
Child n → parent

(n - 1 / 2)

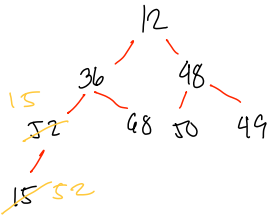
(-1 == 0 / 2 = 0

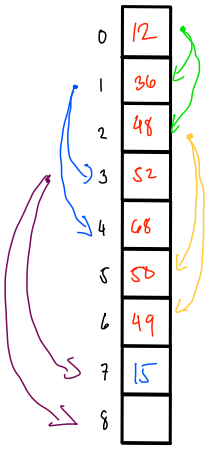
2-1 == 1 / 2 = 0

We will insert to the last index

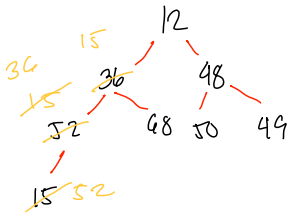


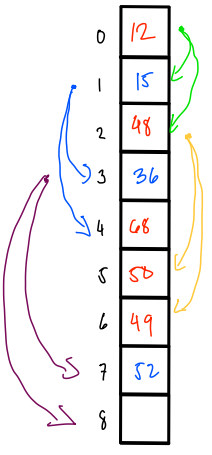
swapped





Binary Heap O(log n)





Insert

1.) insert last node

2.) bubble up P < C

3.) last ++;

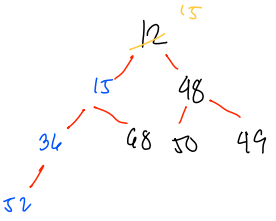
How can we do deletion

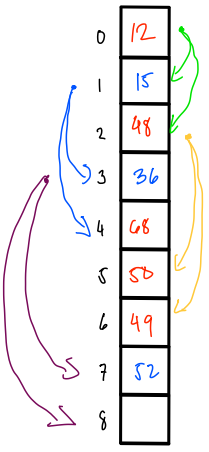
Delete

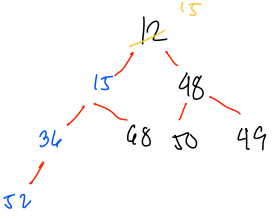
1.) start -> 0

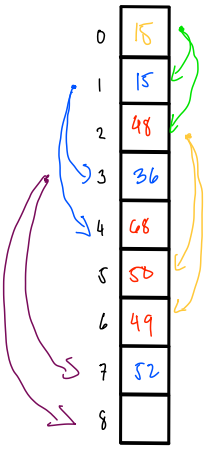
2.) L < R

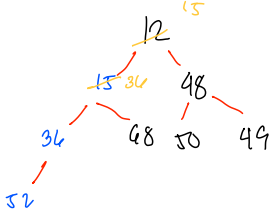
3.) last --

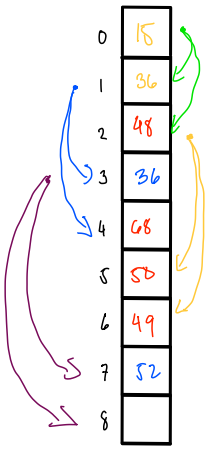


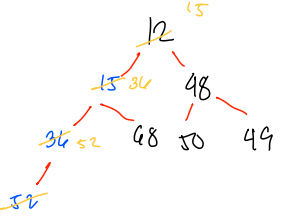


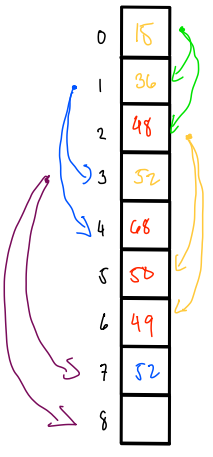












minHeap is what we use for our priority queue

Heapsort

will be discussed on Oct. 26

Heap	index 0	P.O.T property	Operations	Heap sort
minHeap	smallest value	Parent ≤ child	insertLast delMin (deleteFirst)	Descending order
maxHeap	largest value	Parent ≥ child	insertLast delMax (delFirst)	Ascending order

Oct. 26, 2020

ANNOUNCEMENT

Nov. 21 - 26 — async — week miss
- attendance is required

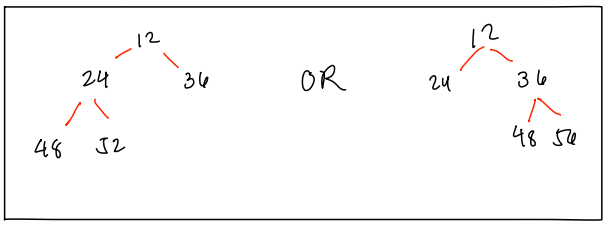
23 & 24 - may sub. prof

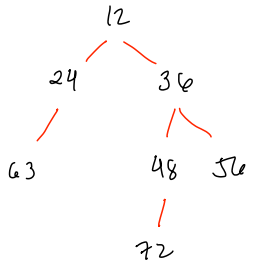
Oct. 28 — review
Nov. 4 — talk about trees

Priority Queue — will have longer extension deadline
↳ [AA] - previous exam question

REVIEW

BINARY HEAP min Heap max Heap



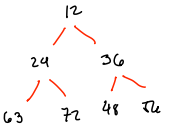


Array format

⇒ you can delete anywhere per depot
no updated any parent child relationship
from deletion or delmax. P < C

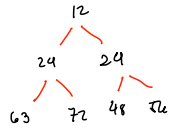
PRIORITY QUEUE → using at as queuing

P.O.T. → Partially Ordered Tree
↳ 1) always start at the left
↳ 2) balanced
↳ every slot no fill up



	BH	PQ	Property	
Insertion			minHeap	P < C
Deletion	any node	root node ↳ top most node	maxHeap	P > C

Q: Can PQ have maxHeap? It can but mostly we use minHeap



Q: In terms of insertion,

	BH	PQ	Property	
Insertion	last	last (always swap up)	minHeap	P < C
Deletion	any node	root node (always swap down) ↳ top most node	maxHeap	P > C

HEAP SORT

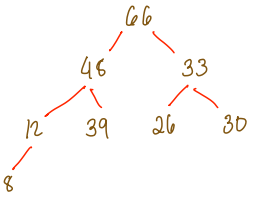
→ what if you can do it in terms of log N notation

Heapsort

will be discussed on Oct. 26

Heap	index 0 first	P.O.T property	Operations	Heap Sort
minHeap	smallest value	Parent ≤ child	insertLast delMin (deleteFirst)	Descending order
maxHeap	largest value	Parent ≥ child	insertLast delMax (delFirst)	Ascending order

maxHeap → HeapSort



From this to → this

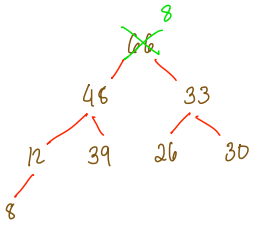
0	66
1	48
2	33
3	12
4	39
5	26
6	30
7	8

8
12
26
30
33
39
48
66

SORTED ARRAY

last = 6
temp = 66

maxHeap → HeapSort

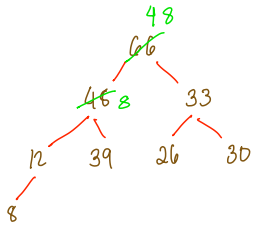


0	~~66~~
1	48
2	33
3	12
4	39
5	26
6	30
7	8

heapSort loop {
1.) temp = BH[0];
2.) delMax
↳ last --
↳ swap down

last = 6
temp = 66

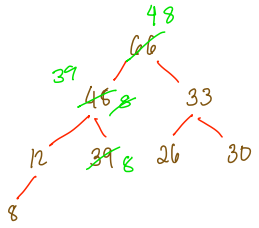
maxHeap → HeapSort



0	48
1	8
2	33
3	12
4	39
5	26
6	30
7	8

heapSort loop {
1.) temp = BH[0];
2.) delMax
↳ last --
↳ swap down

maxHeap -> HeapSort

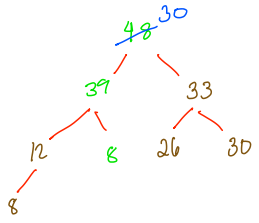


last = 6
temp = 66

0	48
1	39
2	33
3	12
4	8
5	26
6	30
7	8

heapSort loop {
1.) temp = BH[0];
2.) delMax
↳ last --
↳ swap down

maxHeap -> HeapSort

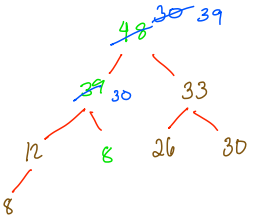


last = 5
temp = 66

0	~~48~~ 30
1	39
2	33
3	12
4	8
5	26
6	30
7	66

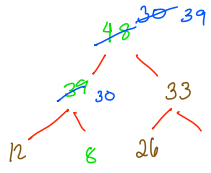
heapSort loop {
1.) temp = BH[0]; //48 ~~null~~ node
2.) delMax
↳ last --
↳ swap down
3.) BH[last + 1] = temp;

maxHeap -> HeapSort



0	~~30~~
1	39
2	33
3	12
4	8
5	26
6	30
7	66

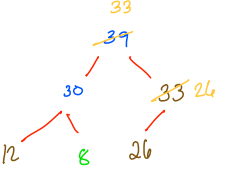
maxHeap -> HeapSort



last = 4
temp = 39

0	39
1	30
2	33
3	12
4	8
5	26
6	48
7	66

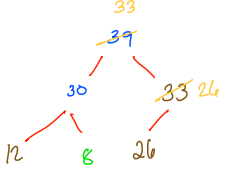
maxHeap -> HeapSort



last = 4
temp = 39

0	~~39~~
1	30
2	33
3	12
4	8
5	26
6	48
7	66

maxHeap -> HeapSort

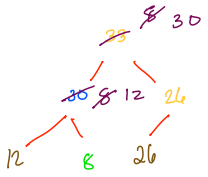


last = 4
temp = 39

0	33
1	30
2	~~26~~
3	12
4	8
5	26
6	48
7	66

maxHeap → HeapSort

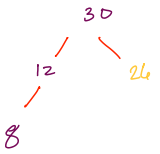
last = 3
temp = 33



0	~~8~~
1	30
2	~~26~~
3	12
4	8
5	26
6	48
7	66

maxHeap → HeapSort

last = 3
temp = 33



0	30
1	12
2	~~26~~
3	8
4	33
5	39
6	48
7	66

8
12
26
30
33
39
48
66

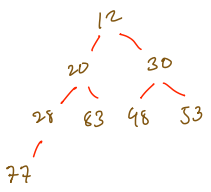
last = -1
↳ last must be 7 again

ascending order

minHeap → descending order

~~maxHeap~~ → HeapSort

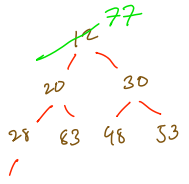
p < c



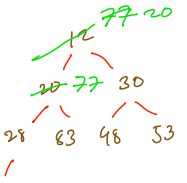
0	12	83
1	20	77
2	30	53
3	28	48
4	83	30
5	48	28
6	53	20
9	77	12

```
heapSort Imp {
1.) temp = BH[0];
2.) delMin;
      ↳ last --
      ↳ swapdown;
3.) BH[last+1] = temp
}
```

last = 6
temp = 12



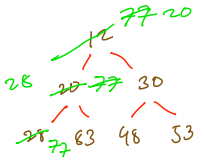
0	~~12~~ 77	83
1	20	77
2	30	53
3	28	48
4	83	30
5	48	28
6	53	20
9	77	12



last = 6
temp = 12

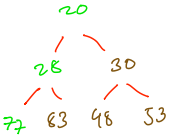
0	~~77~~ 20	83
1	20	77
2	30	53
3	28	48
4	83	30
5	48	28
6	53	20
9	77	12

last = 6
temp = 12

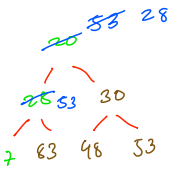


0	~~77~~ 20	83
1	20	77
2	30	53
3	28	48
4	83	30
5	48	28
6	53	20
9	77	12

last = 6
temp = 12

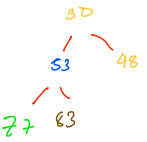


0	20	83
1	28	77
2	30	53
3	77	48
4	83	30
5	48	28
6	53	20
9	12	12



last = 5
temp = 20

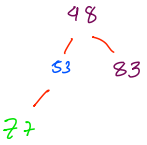
0	28	83
1	53	77
2	30	53
3	77	48
4	83	30
5	48	28
6	20	20
7	12	12



last = 4
temp = 28

0	30	83
1	53	77
2	48	53
3	77	48
4	83	30
5	28	28
6	20	20
7	12	12

last = 3
temp = 30



0	48	83
1	53	77
2	83	53
3	77	48
4	30	30
5	28	28
6	20	20
7	12	12

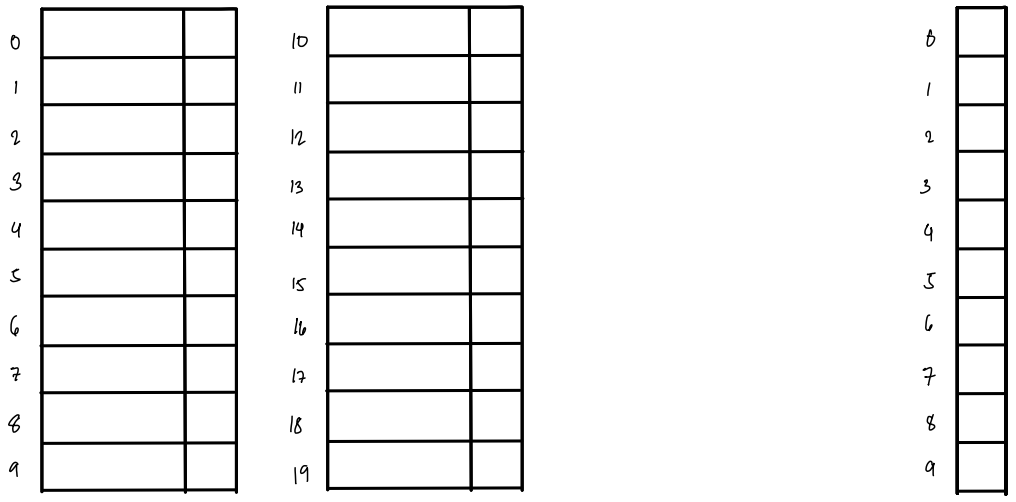
0		
1		
2		
3		
4		
5		
6		
7		
8		
9		

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

- 2 Assignment on Paper submit on Wednesday.
- 1 Assignment on CodeChum submit on Wednesday.

Oct. 28, 2024

Review on Hashing Activity (Problems 1 - 5) in Canvas

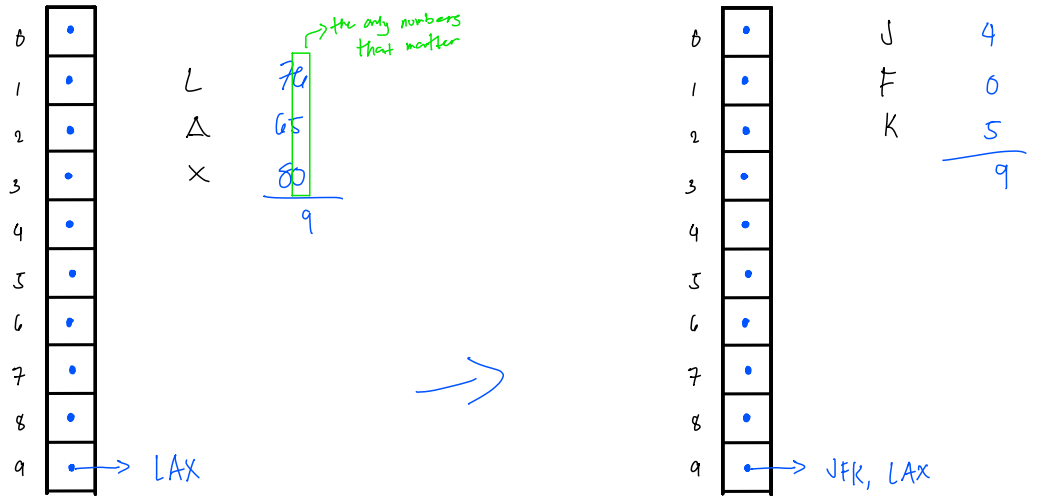


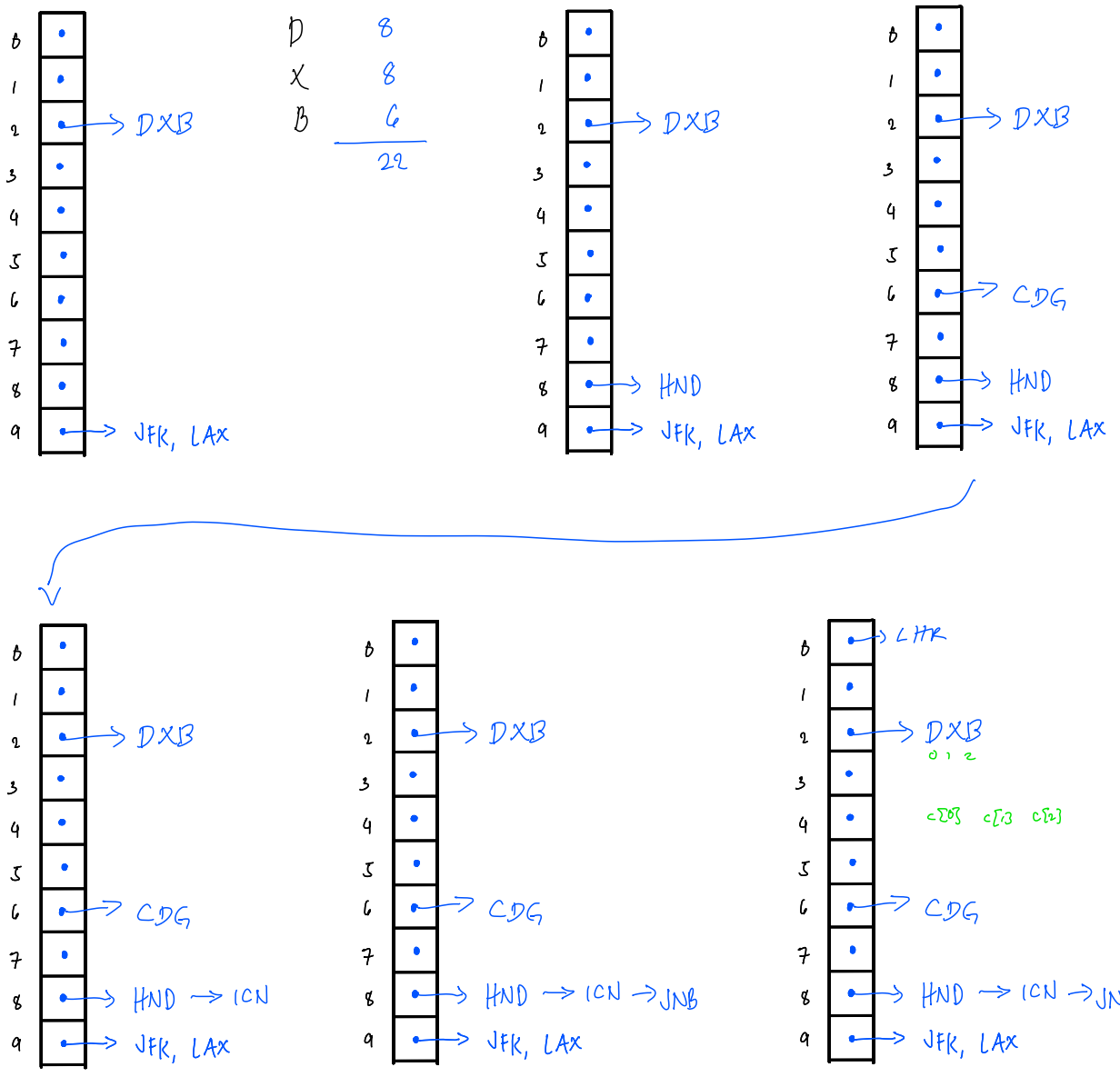
Problem 1: Airport Code Arrivals

A = 65 ~ Z = 90

```
int Hash(string c){
    return (c[0] + c[1] + c[2]) % 10;
}
```

(c[0] → 7, c[1] → 11, c[2] → 13)

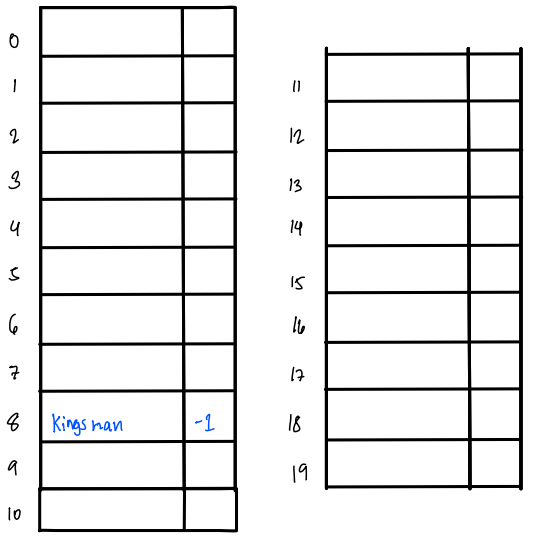


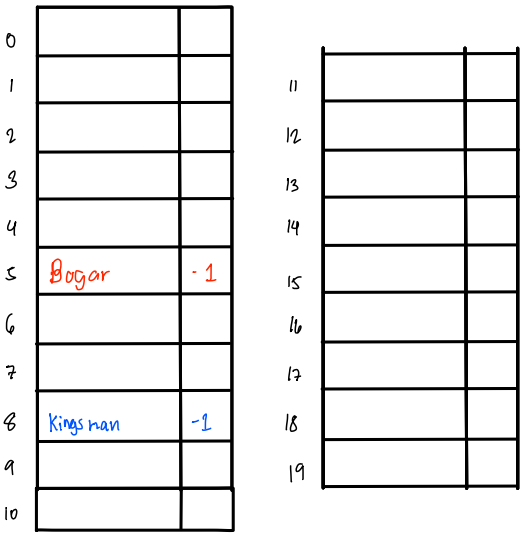


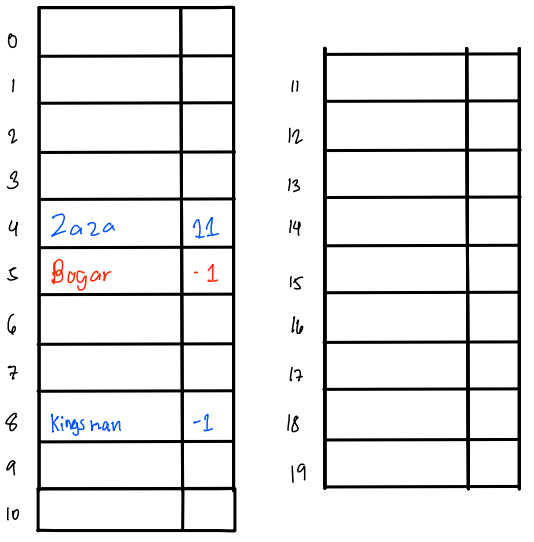
- to complicated na in terms of index 8 & 9.

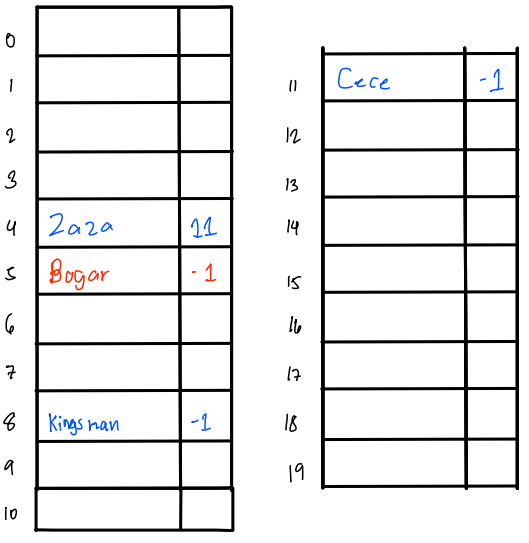
PROBLEM 2: Gaming Lobby Usernames

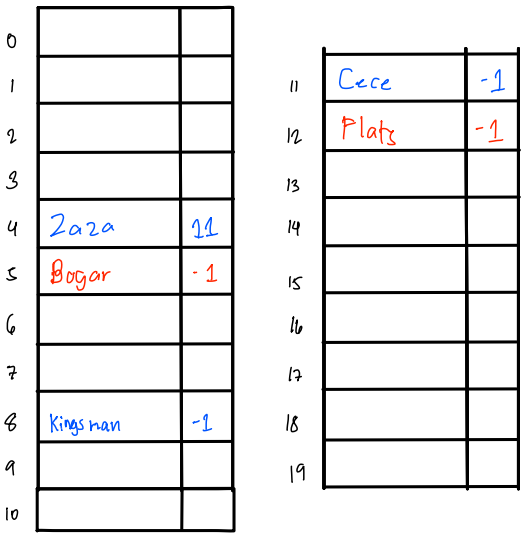
int Hash(string u) {

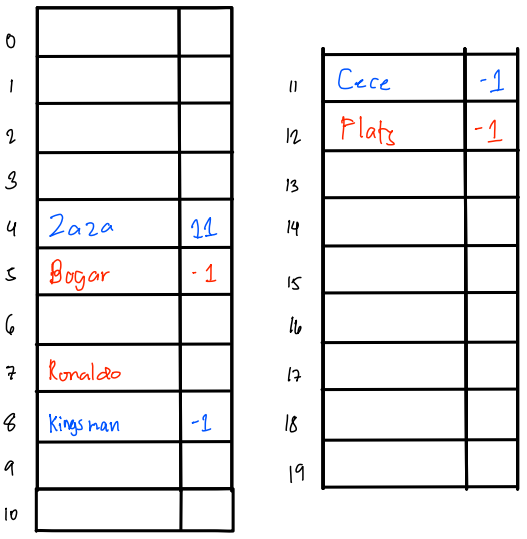


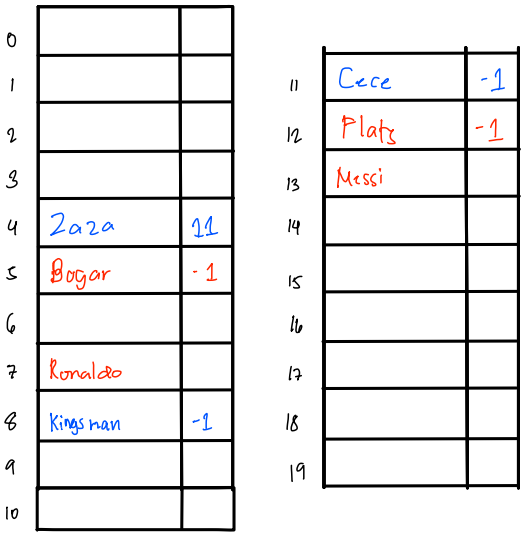


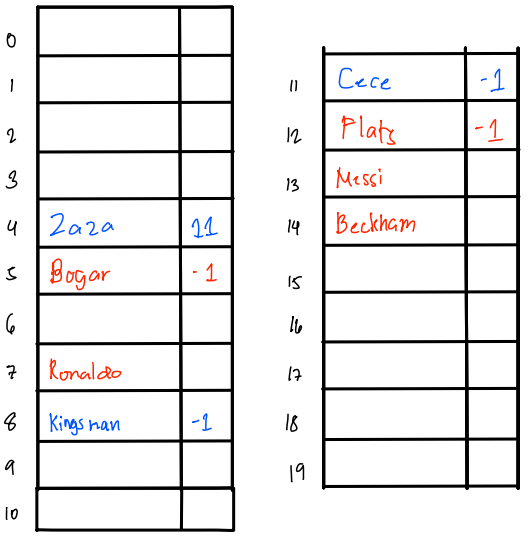


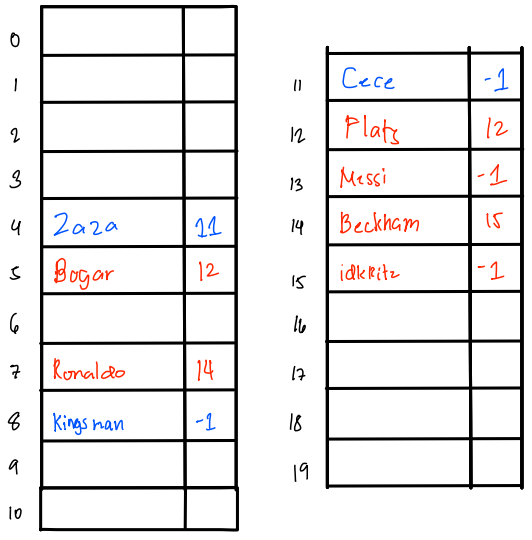


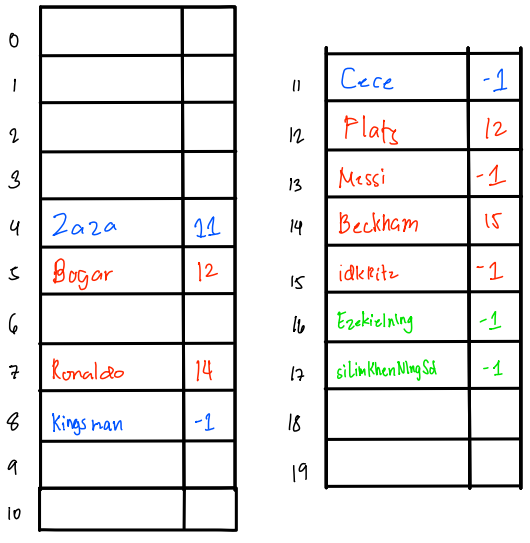

0
1
2
3
4 Zaza 11
5 Bogar -1
6
7
8 Kingsman -1
9
10
11 Cece -1
12 Platz -1
13
14
15
16
17
18
19

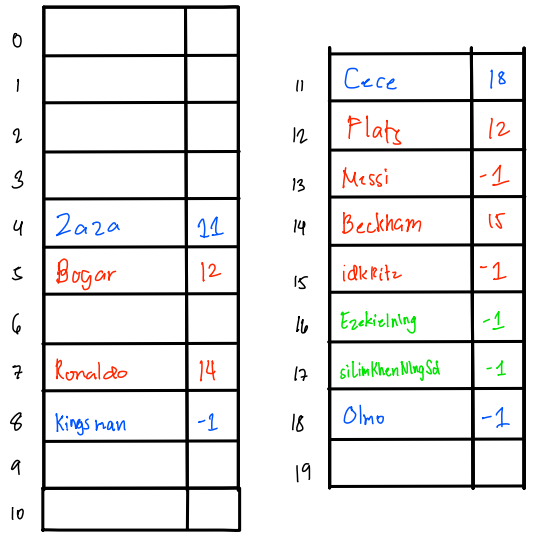

0
1
2
3
4 Zaza 11
5 Bogar -1
6
7 Ronaldo
8 Kingsman -1
9
10
11 Cece -1
12 Platz -1
13
14
15
16
17
18
19

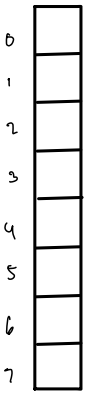

0
1
2
3
4 Zaza 11
5 Bogar -1
6
7 Ronaldo
8 Kingsman -1
9
10
11 Cece -1
12 Platz -1
13 Messi
14
15
16
17
18
19


0
1
2
3
4 Zaza 11
5 Bogar -1
6
7 Ronaldo
8 Kingsman -1
9
10
11 Cece -1
12 Platz -1
13 Messi
14 Beckham
15
16
17
18
19


0
1
2
3
4 Zaza 11
5 Bogar 12
6
7 Ronaldo 14
8 Kingsman -1
9
10
11 Cece -1
12 Platz 12
13 Messi -1
14 Beckham 15
15 idkRitz -1
16
17
18
19


0
1
2
3
4 Zaza 11
5 Bogar 12
6
7 Ronaldo 14
8 Kingsman -1
9
10
11 Cece -1
12 Platz 12
13 Messi -1
14 Beckham 15
15 idkRitz -1
16 EzekielNing -1
17 siLimKhenNingSid -1
18
19


0
1
2
3
4 2a2a 11
5 Bogar 12
6
7 Ronaldo 14
8 Kingsman -1
9
10
11 Cece 18
12 Platz 12
13 Messi -1
14 Beckham 15
15 idkRitz -1
16 Ezekielning -1
17 siLimKhenNingSd -1
18 Olmo -1
19


0
1
2
3
4
5
6
7

PROBLEM 3: Cafe Digital Menu

```
int Hash(string u) {

    traverse
      if(acionAEIOU)

    switch (u[x])
      case 'a': case 'e': case ' ', ... );
                count++;
      default:  break;
                break;
```

0	
1	
2	
3	
4	
5	
6	
7	

8	
9	
10	
11	
12	
13	
14	
15	

3

0		
1		
2	Latte	-1
3		
4		
5		
6		
7		
8		
9		
10		

11		
12		
13		
14		
15		
16		
17		
18		
19		

0		
1		
2	Latte	-1
3	Espresso	11
4		
5		
6		
7		
8		
9		
10		

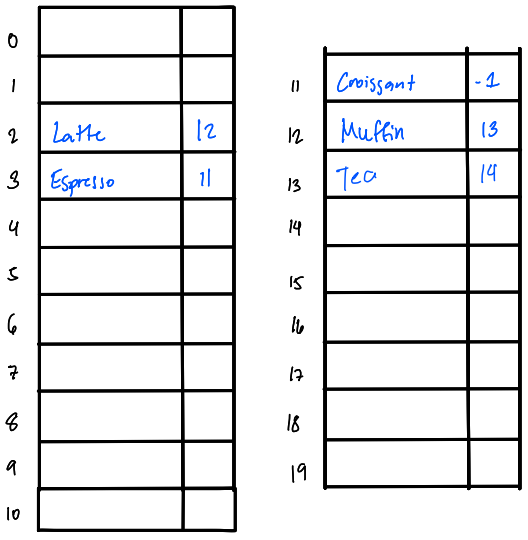
11		
12		
13		
14		
15		
16		
17		
18		
19		

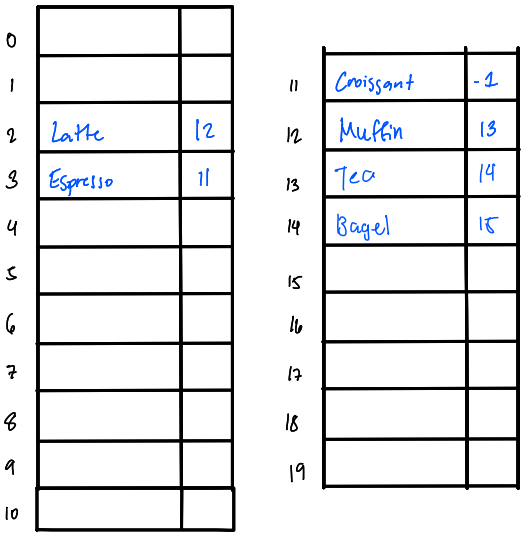
0		
1		
2	Latte	-1
3	Espresso	11
4		
5		
6		
7		
8		
9		
10		

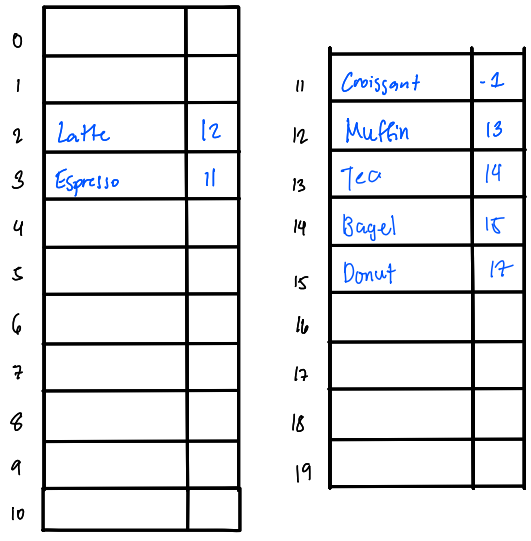
11	Croissant	-1
12		
13		
14		
15		
16		
17		
18		
19		

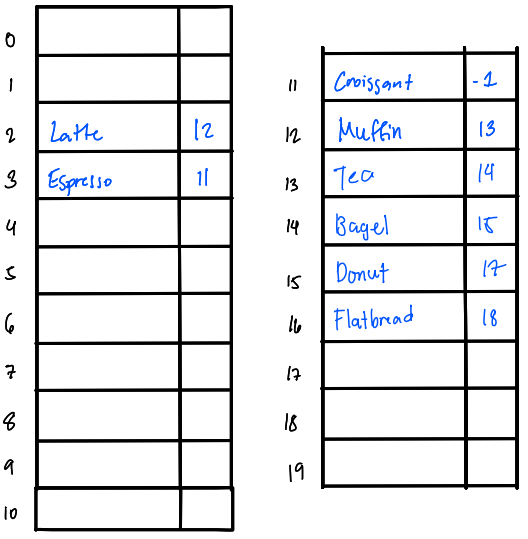
0		
1		
2	Latte	12
3	Espresso	11
4		
5		
6		
7		
8		
9		
10		

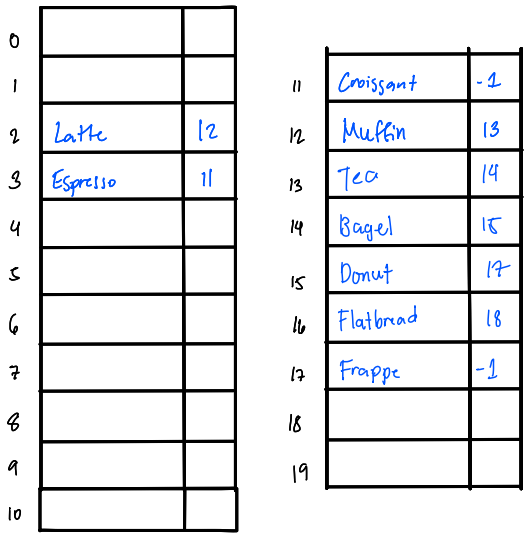
11	Croissant	-1
12	Muffin	13
13		
14		
15		
16		
17		
18		
19		


0
1
2 Latte 12
3 Espresso 11
4
5
6
7
8
9
10
11 Croissant -1
12 Muffin 13
13 Tea 14
14
15
16
17
18
19


0
1
2 Latte 12
3 Espresso 11
4
5
6
7
8
9
10
11 Croissant -1
12 Muffin 13
13 Tea 14
14 Bagel 15
15
16
17
18
19


0
1
2 Latte 12
3 Espresso 11
4
5
6
7
8
9
10
11 Croissant -1
12 Muffin 13
13 Tea 14
14 Bagel 15
15 Donut 17
16
17
18
19


0
1
2 Latte 12
3 Espresso 11
4
5
6
7
8
9
10
11 Croissant -1
12 Muffin 13
13 Tea 14
14 Bagel 15
15 Donut 17
16 Flatbread 18
17
18
19


0
1
2 Latte 12
3 Espresso 11
4
5
6
7
8
9
10
11 Croissant -1
12 Muffin 13
13 Tea 14
14 Bagel 15
15 Donut 17
16 Flatbread 18
17 Frappe -1
18
19

. . .

PROBLEM 4: Social Media Hashing

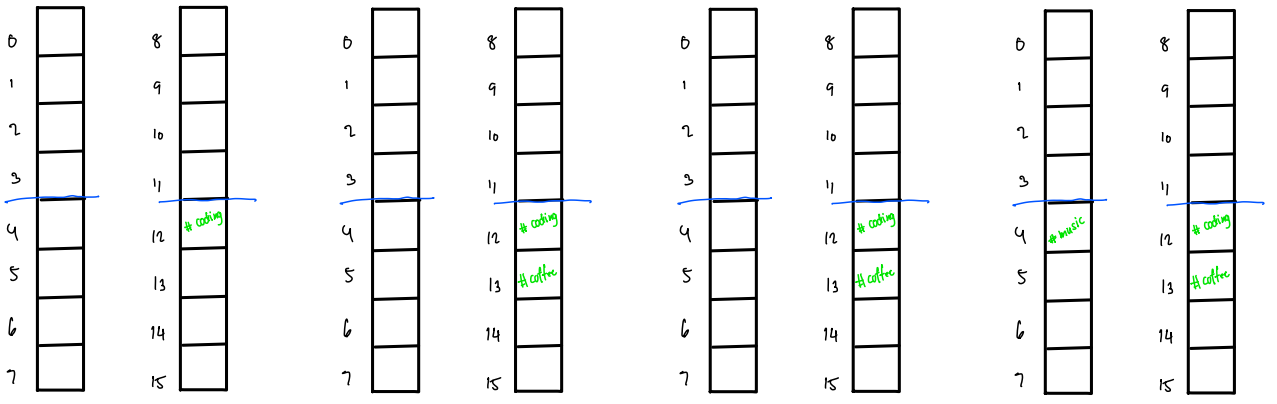
```
int Hash (string h) {
    return ((h[2] - 'a') % 16)
           / 4
}
```

↳ divide by 4 since Bucket range of 4.

$16\overline{)15}$ 0 R. 15

$15/4 = 3._$

$16\overline{)21}$ (1) -16 (5) OR -21 -16 (5)



- ma ommitted si#marvelrivals

It's more efficient to insert at first than insertLast because it is LinkedList.

REVIEW for EXAM 2

2.)

0	1	1	0	1	0	0	0
128	64	32	16	8	4	2	1
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

104

5.) B[5] = 1 ?

7.) computer used is for space efficiency

8.) int Hash (string h) {

↳ key

↳ the value you pass to the hash() that will generate a hash index

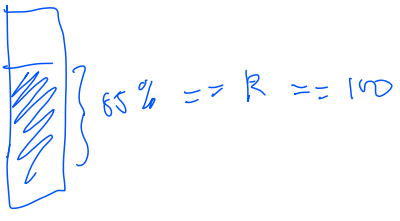
Hash table → before the simulation

10.) i is 8 because of 8 bit type

11.)

100 ÷ .85 = 117.65 from ceiling it is now 118

R == N

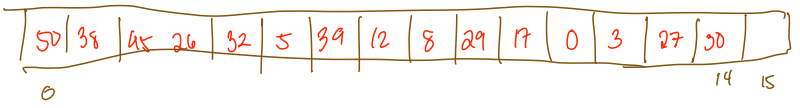


14.)

C is false, it should be O(N)

B is O(N)

Canvas



Nov. 4, 2024

Quizzes:
- Bit vector
- heap sorting

E2 PREVIOUS. not to be used (60 points)
- codeChum: heap & dictionary

Item 4: Encoding
- From bits to array to word

- shifting means multiplication by 2

7 char
15 short int
31 sorting

Kabisanghon ko lmao XD

- PQ -> inp is binary ordered tree
which is derived from sorting

- Syn on expect may hashing sa PQ (Priority Queue)

Coverage
- Bit Vector, Dictionary, PQ => ADT Sets

can be array/compound ✓

- 3 coding items only

ACTIVITY : to be done inside the classroom.

For each number, supply the ff:

a.) binary of the num.

b.) binary of the result.

c.) code snippet for the result.

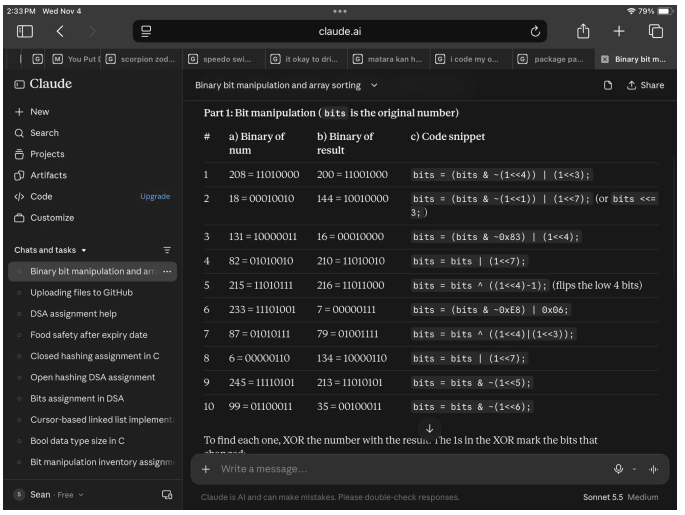
ex. 239 to 175

a) 239 to 1110 1111

b) 175 to 1010 1111

c) bits = bits & (~(1 << 6))

note: all numbers are



1.) 208 to 200

a.) 208 is 1101 0000

b.) 200 is 1100 1000

c.) bits = bits | (~(1 << 4))

```
  0001 0000
| 1100 1000
-----------
  1101 0000
```

2.) 18 to 144

a) 18 is 0001 0010

b) 144 is 1001 0000

c) bits = bits & (~(1 << 1))

```
  0000 0010
& 1001 0000
-----------
  1001 0000
```

3) 131 to 16

a) 131 is 1000 0011

b) 16 is 0001 0000

c) bits = bits ~~(1 << 7)~~ (1<<4) ~~(1<<1) (1<<0)~~

~~1000 0000~~

0001 0000 bits = bits (1 << 4)

~~0000 0010~~

~~0000 0001~~

4) 82 to 210

a) 82 is 0101 0010

b) 210 is 1101 0010

c) bits = bits | (1<< 7);

```
  1000 0000
| 0101 0010
-----------
  1101 0010
```

5.) 215 to 216 1<<3 bits | (1<<3);

a) 215 is 1101 0111 (or) 0000 1000 / 1101 1000 / 1101 1000

b) 216 is 1101 1000

c) bits = bits (1 << 2) (1<<1) (1<<0)

(or)

```
  0000 0100
| 0000 0010
-----------
  0000 0001
```

or

```
  0000 0111
| 1101 1000
-----------
  1101 1111
```

& 0010 0000 1<<5

& 1101 1000

1111 1000

6.) 233 to 7

a) 233 is 1110 1001

b) 7 is 0000 0111

c) bits = bits | (1<<0) | (1<<2);

OR 0000 0001

~~0000 1000~~

0000 0100

OR 0000 0101

0000 0111

0000 0111

7.) 87 to 79

a) 87 is 0101 0111

b) 79 is 0100 1111

c.) bits = bits | (1<<6)

OR 0100 0000 0000 0010 OR

0100 1111 0100 1111

0100 1111

8.) 6 to 134

a) 6 is 0000 0110

b) 134 is 1000 0110

c) bits = bits | (1<<7)

OR 1000 0000

1000 0110

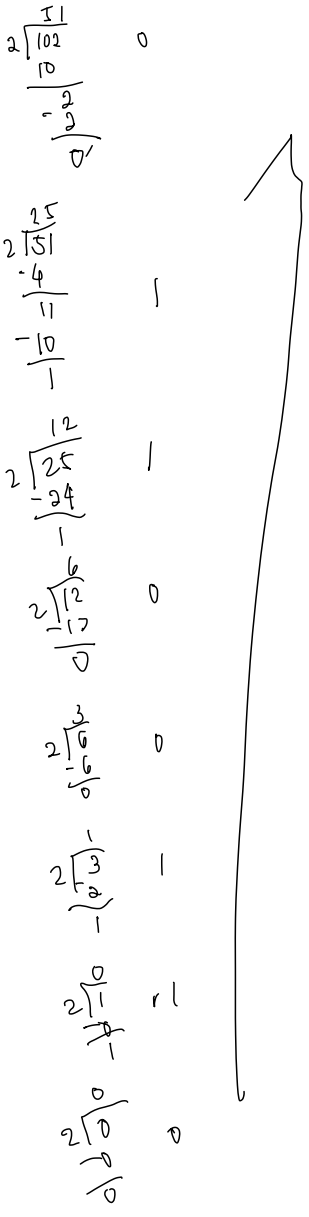
1000 0110

128 64 32 16 8 4 2 1
0 0 0 0 0 0 0 0

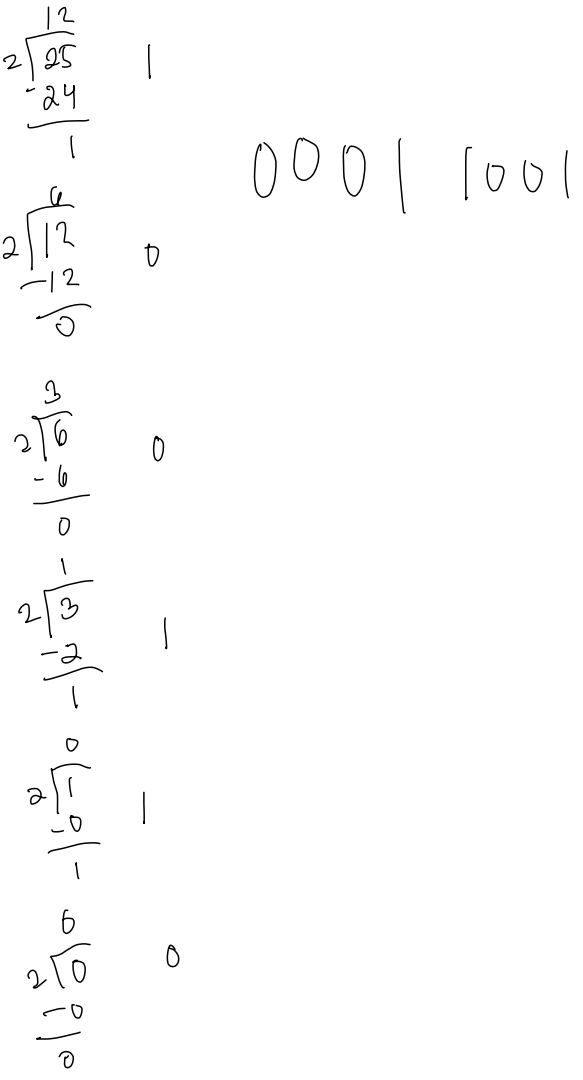
SETS

Procedure on Left Shift << - Multiplication by power of 2

1.) a) 102 is 0 1 1 0 0 1 1 0



0 1 1 0 0 1 1 0



0 0 0 1 1 0 0 1

(102 << 1)

128 64 32 16 8 4 2 1

0 1 1 0 0 1 1 0 → 1 1 0 0 1 1 0 0

OR 0 0 0 0 0 0 1 0

0 1 1 0 0 1 1 0

64 32 4 2 1 0 1 0

1 64 32 4 2 = 102

1 1 128 64 12 = 204

128 64 32 16 8 4 2 1

Ways To Get From Decimal To Binary

Decimal
↙
76 binary →

76
- 64
12
- 8
4
- 4
0

128	64	32	16	8	4	2	1
0	1	0	0	1	1	0	0
1	1	0	1	0	1	1	1

215 →

215
- 128
87
- 64
23
- 16
7
- 4
3
- 2
1
- 1
0

Procedure on Left.

Dec. Bin. <<

102 → 0110 0110 → 1100 1100 = 204

128	64	32	16	8	4	2	1
1	1	0	0	1	1	0	0
0	0	1	1	0	0	1	1

1 1
128
64
12
204

Procedure on Right Shift

Dec. Bin. >>

102 → 0110 0110 → 0011 0011 = 51

1
32
16
3
51

GIVEN;

```
x = 6;      // 0000 0110
y = 12;     // 0000 1100
z = 0;      // 0000 0000
```

XOR (^)

```
z = x ^ y
      0000 0110
      0000 1100
      ---------
z =   0000 1010  = 10
```

Left Shift (<<)

```
z = x << 1;
      0000 0110
z =   0000 1100  = 12
z = x << 2;
      0000 0110
z = 0001 1000  = 24
```

Right Shift (>>)

```
z = x >> 1;
      0000 0110
z =   0000 0011  = 3
z = x >> 2;
      0000 0110
z = 0000 0001  = 1
```

CodeChum
BITS

ITEM 1) Bit Vector Activity

```
branch_A = 0;      // 0000 0000
branch_B = 0;

branch_A |= (1 << 7);   // 1000 0000
branch_A |= (1 << 5);   // 0010 0000
branch_A |= (1 << 4);   // 0001 0000
branch_A |= (1 << 2);   // 0000 0100
branch_A |= (1 << 0);   // 0000 0001
```

or

```
1000 0000
0010 0000
0001 0000
0000 0100
0000 0001
1011 0101
```

```
int i = 1 << (sizeof(products)*8-1);
```

- sizeof(products) → gives size in bytes
- *8 → converts bytes to bits, so you get 8.
- -1 → gives 7, position of highest bit.

1 byte = 8 bits

1 * 7

products = 1011 0101

[7th] i = 1000 0000 ← start at 7

products & i = 1000 0000 → nonzero → bit is ON

then i >> 1;

(1) 1000 0000

(2) 0100 0000

products = 1011 0101

[6th] i = 0100 0000

products & i = 00000000 → zero → bit 6 is OFF

unsigned char = 1 byte = 8 bits
short = 2 bytes = 16 bits
int = 4 bytes = 32 bits

I tried but I feel lack
I'm not doing enough.

```
while (i > 0) {
    if (products & i) {
        printf("1");
    }
    i = i >> 1;
    prod--;
}
printf("\n");
```

```
i = 1 << (sizeof(products)*8-1);
```

↳ 1 << (1 * 8 - 1) = (8 - 1) = 7

i = 7 ✓ prod

```
i = i >> 1;
```

↳ i = 0100 0000 [6]

print:
prod_7,

products = 1011 0101

[7] i = 1000 0000

products & i = 1000 0000 = ON = print prod_7

```
while (i > 0) {
    if (products & i) {
        printf(" prod_%d ", prod);
    }
    i = i >> 1;
    prod--;
}
printf("\n");
```

products = 1011 0101

(products & i)

& products = 1011 0101
  i = 1000 0000 [128]
products & i = 1000 0000 (128) = ON

& products = 1011 0101
  i = 0100 0000 [64]
products & i = 0000 0000 (0) = OFF

& products = 1011 0101
  i = 0010 0000 [32]
products & i = 0010 0000 (32) = ON

& products = 1011 0101
  i = 0001 0000 [16]
products & i = 0001 0000 (16) = ON

& products = 1011 0101
  i = 0000 1000 [8]
products & i = 0000 0000 (0) = OFF

& products = 1011 0101
  i = 0000 0100 [4]
products & i = 0000 0100 (4) = ON

& products = 1011 0101
  i = 0000 0010 [2]
products & i = 0000 0000 (0) = OFF

i = 1 << (sizeof(products) * 8 - 1); // 7
  1 << (1 byte) * 8 - 1
  1 << 7; = 0010 0000 (32) = 0000 0100 (4)
i = 1000 0000 = 128 = 0001 0000 (16) = 0000 0010 (2)
  = 0100 0000 = 64 = 0000 1000 (8) = 0000 0001 (1)
prod = sizeof(products) * 8 - 1; → = 0000 0000 (0)
  = (1 byte) * 8 - 1
prod = 7

i > 0

(1) i = 128 ✓
(2) i = 64 ✓
(3) i = 32 ✓
(4) i = 16 ✓
(5) i = 8 ✓
(6) i = 4 ✓
(7) i = 2 ✓
(8) i = 1 ✓
(9) i = 0 ✗ ←

i = i >> 1;
  = 0100 0000 (64)
  = 0010 0000 (32)
  = 0001 0000 (16)
  = 0000 1000 (8)
  = 0000 0100 (4)
  = 0000 0010 (2)
  = 0000 0001 (1)
→ = 0000 0000 (0)

prod--;
  prod = 6
  prod = 5
  prod = 4
  prod = 3
  prod = 2
  prod = 1
  prod = 0
→ prod = -1

printf:
prod_7 prod_5 prod_4 prod_2 prod_0

print here →

& products = 1011 0101
  i = 0000 0001 [1]
products & i = 0000 0001 (1) = ON

^ - XOR

p	q	p ^ q
1	1	0
1	0	1
0	1	1
0	0	0

```
branch_A |= (1 << 7); // 1000 0000
branch_A |= (1 << 6); // 0100 0000
branch_A |= (1 << 4); // 0001 0000
branch_A |= (1 << 3); // 0000 1000
branch_A |= (1 << 0); // 0000 0001
                  A=  1101 1001
```

128 + 64 + 16 + 8 + 1 = 217

```
branch_B |= (1 << 7); // 1000 0000
branch_B |= (1 << 6); // 0100 0000
branch_B |= (1 << 5); // 0010 0000
branch_B |= (1 << 2); // 0000 0100
branch_B |= (1 << 0); // 0000 0001
                  B=  1110 0101   = 229
```

128 + 64 + 32 + 4 + 1 = 229

```
  1101 1001   A
| 1110 0101   B
-------------
  1111 1101   -> output
```

```
  A      1101 1001
& ~B     0001 1010
-------------------
  0001 1000   prods 4 & 3
```

```
moveInventory() {
    unsigned char toMove = *branchA & ~(*branchB);

    *branchB |= toMove;
    *branchA &= ~toMove;

    printf("Prods moved successfully\n");
}
```

```
toMove = *branchA & ~(*branchB);
    A =    1101 1001
& ~B =     0001 1010
-------------------
toMove =   0001 1000
```

```
*branchB |= toMove;
branchB =  1110 0101
toMove =   0001 1000
-------------------
           1111 1101
           7 6 5 4 3 2 1 0
```

```
*branchA &= ~toMove;
branchA =  1101 1001
& ~toMove = 1110 0111
-------------------
            1100 0001
            7 6     0
```

```
branch_A |= (1 << 5); //  0010 0000
branch_A |= (1 << 4); //  0001 0000
branch_A |= (1 << 1); //  0000 0010
                branch_A = 0011 0010   (50)
```

```
branch_B |= (1 << 6); //  0100 0000
branch_B |= (1 << 3); //  0000 1000
branch_B |= (1 << 1); //  0000 0010
                branch_B = 0100 1010   (74)
```

```
unavailableProducts () {
    unsigned char result = branchA |= branchB;
```

```
                OR
branch_A =  0011 0010   (50)
branch_B =  0100 1010   (74)
            0111 1010   = 122
```